



南京大學
NANJING UNIVERSITY

五、定时器系统

主讲教师：闫浩
机器人与自动化学院



目录

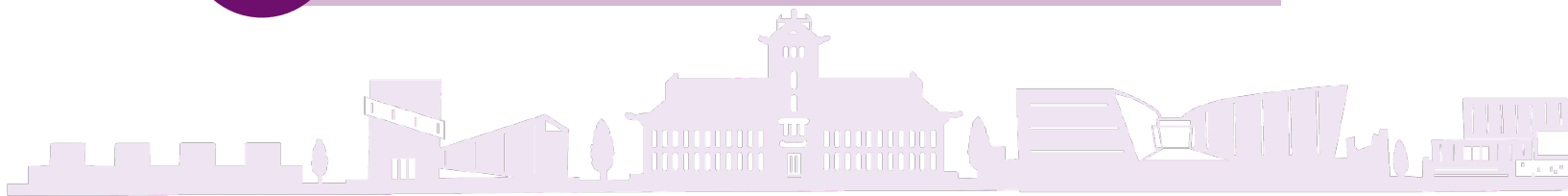
CONTENTS

01 STM32的时钟系统

02 定时器系统简介

03 基本定时功能

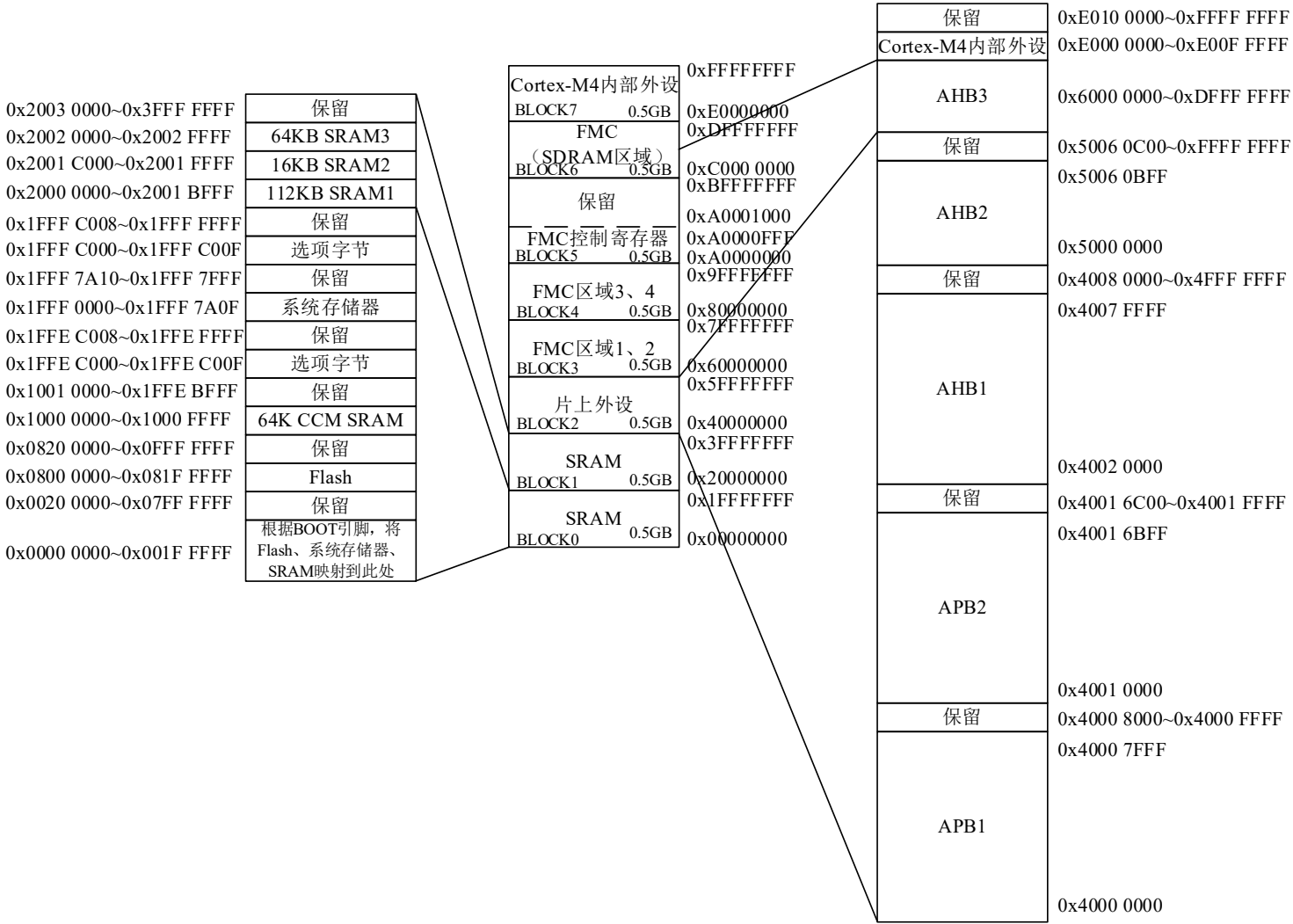
04 捕获/比较功能



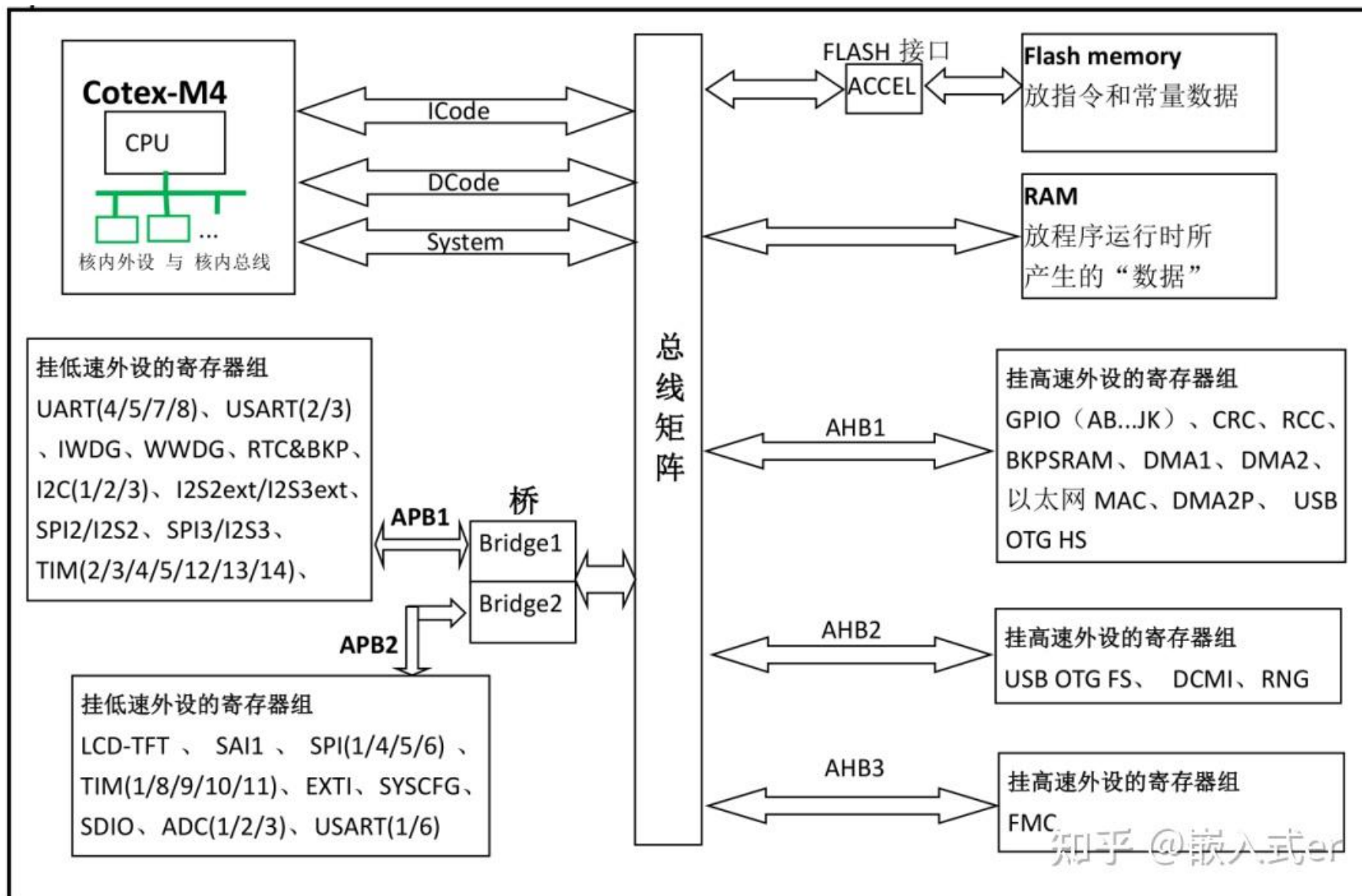
01 STM32的时钟系统

- STM32的各个片上外设通过**APB1、APB2总线**和**AHB1、AHB2总线**连接到内核上。
- 每个总线都有对应的基地址，挂载在相应总线上的外设寄存器的地址，都以相应总线的基地址进行偏移。

AHB，是**Advanced High performance Bus**的缩写，用于高性能模块之间的连接
APB，是**Advanced Peripheral Bus**的缩写，用于低带宽的周边外设之间的连接



STM32的时钟系统



可以类比于计算机的**南北桥**。**南桥总线**上挂接的都是鼠标、键盘这些**慢速**的设备，**北桥**上挂接显卡等**高速**设备。**南桥频率低**，**北桥频率高**。另外，南桥最后也要接到北桥上。这些感觉都类似于APB和AHB。

STM32的时钟系统

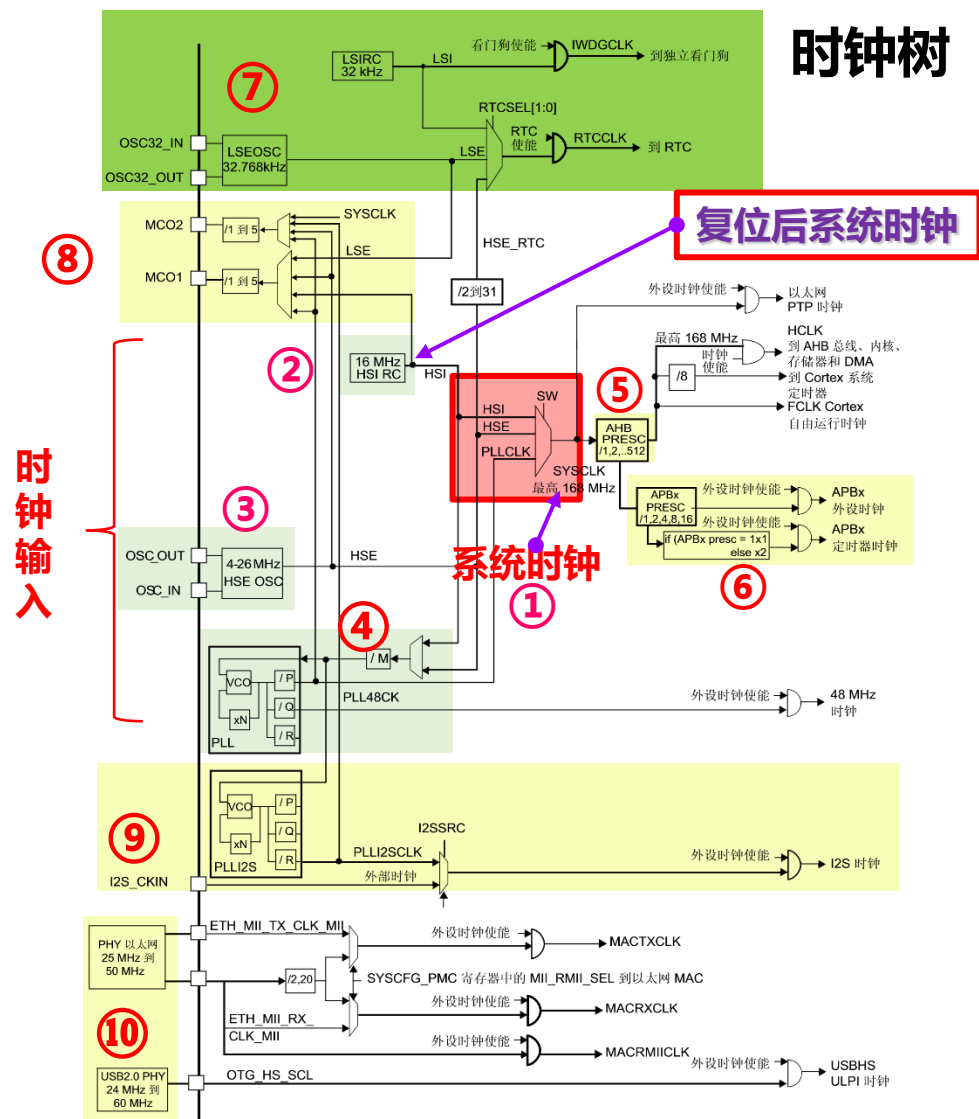
控制器内核、总线、片上外设多数为数字时序逻辑，它们都需要时钟系统的约束，按照一定节拍工作。

复位后，系统时钟已经初始化好
(`system_stm32f4xx.c: SystemInit`) 。

①系统时钟 (SYSCLK) 是核心时钟。

SYSCLK有三个输入选择：② HSI、③ HSE 、④ PLLCLK。

SYSCLK生成总线时钟 ⑤ AHB时钟：HCLK ， AHB时钟生成APB时钟⑥：PCLK1和PCLK2。



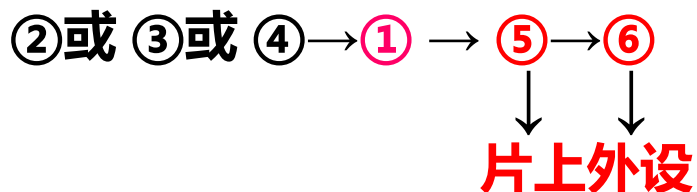
STM32的时钟系统

大部分片上外设挂载在AHB和APB总线上，总线时钟HCLK、PCLK1、PCLK2也就提供了大多数片上外设的工作时钟。

要想使用片上外设，相应程序首先要使能对应总线到片上外设的时钟。

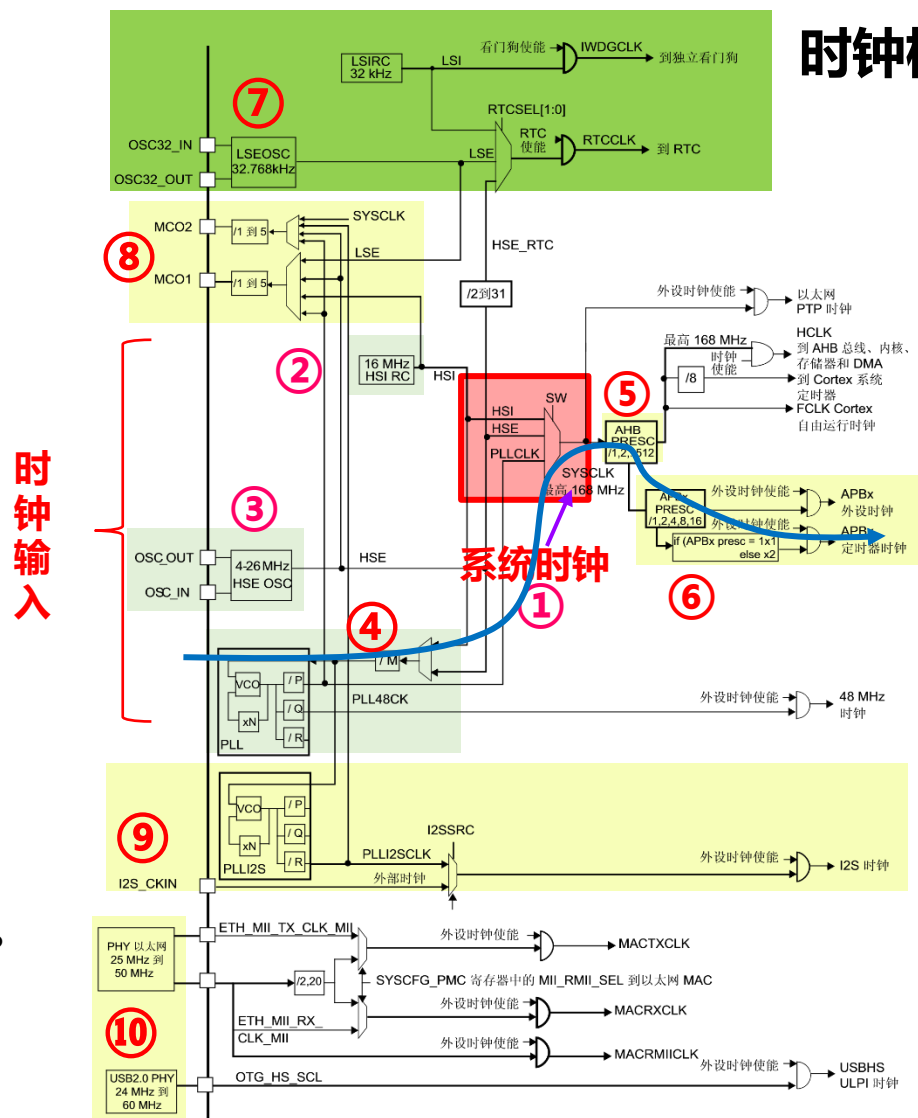
例：

```
RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA, ENABLE);
```



⑦：低速时钟，由外部或内部产生，主要做RTC和看门狗时钟。

⑧：控制器输出时钟。⑨：PLL2生成I2S时钟。⑩：以太网PHY和USB2.0PHY时钟。



STM32F4 系统时钟 (SYSCLK) 可由三种不同的时钟源来驱动:**HSI**、**HSE**、**PLL**



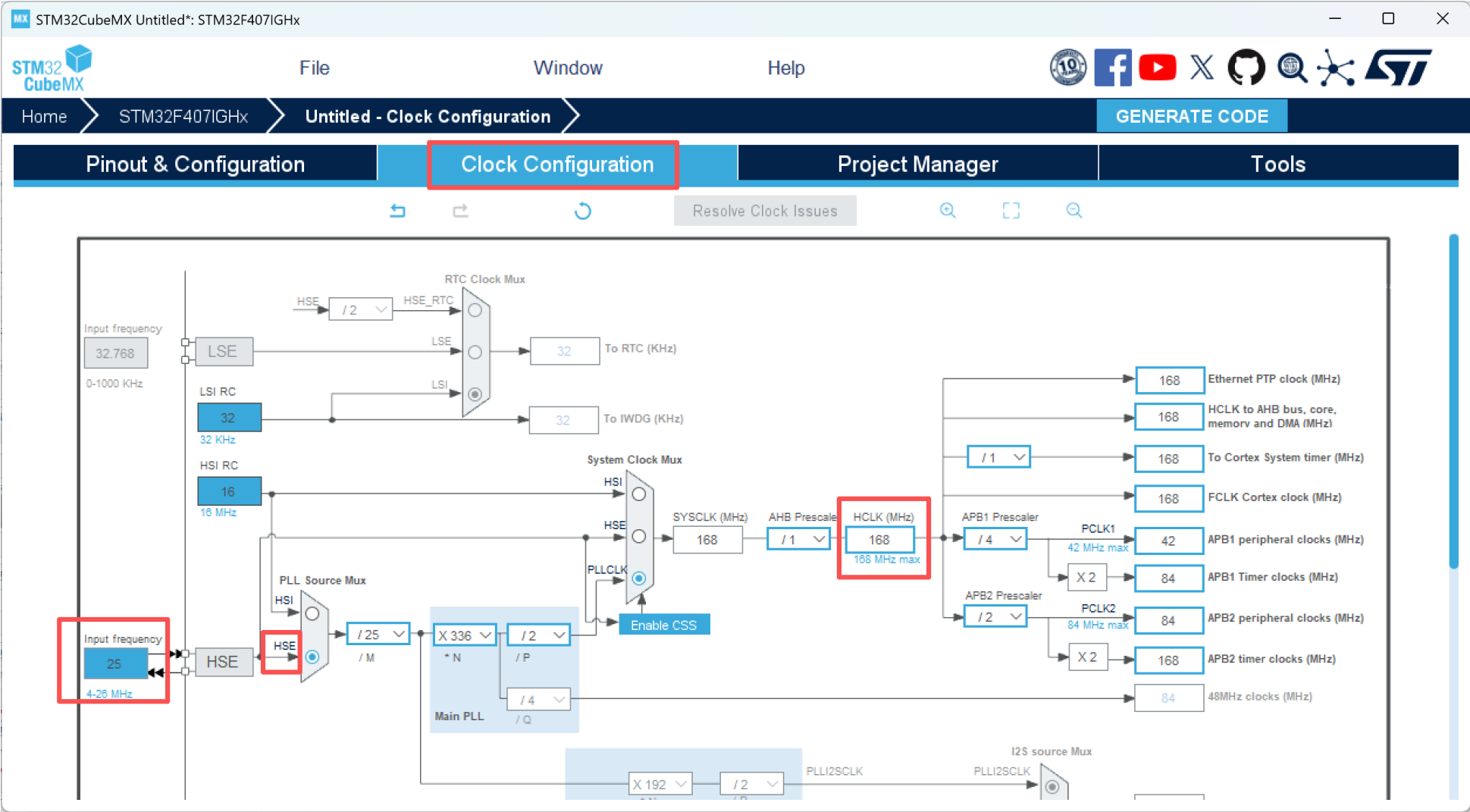
1) **HSI是高速内部时钟**，RC振荡器，精度不高。可以直接作为系统时钟或者用作PLL时钟输入。

2) **HSE是高速外部时钟**，可接石英/陶瓷谐振器，或者接外部时钟源，频率范围为4MHz~26MHz。

3) **PLL为锁相环倍频输出**。STM32F4有两个PLL:

- ◆ 主PLL(PLL)由HSE或者HSI提供时钟信号，并具有两个不同的输出时钟。
- ◆ 专用PLL(PLL12S)用于生成精确时钟，从而在I2S接口实现高品质音频性能。

STM32CubeMX图形化时钟配置界面



02 定时器系统简介

定时器/计数器是MCU基本功能之一，在实际应用中主要用于**精确定时和外部计数**，通常简称为定时器。STM32微控制器提供**基本定时器、通用定时器、高级控制定时器**等多种不同性能的定时器以满足不同场合应用需求。

定时器有很多用途，包括**基本定时功能、生成输出波形**（输出比较、PWM和带死区插入的互补PWM）和**测量输入信号的脉冲宽度**（输入捕获）等。

定时器是对**周期固定**的MCU内部外设**时钟信号**进行**计数**，当达到**计数值**时会产生**中断**，从而达到精确定时的目的。如脉冲信号周期为**1ms**，要产生**1s**精确定时，则可设置定时器从**0**开始向上递增计数，当计数值达到**1000**时触发中断。

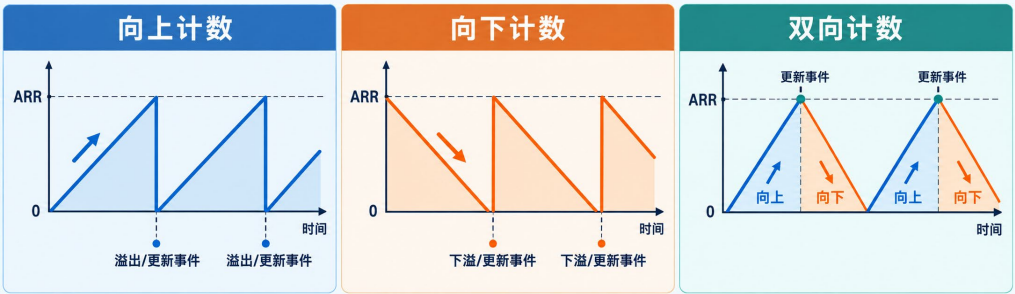
计数器是对**周期不确定的外部脉冲**进行**计数**，通常用于统计**一段时间内外部脉冲个数**，可用于信号频率测量、旋转设备转速测量、流水线工件计数等应用。

本质上，**定时器和计数器都是计数器**，**区别是计数对象不同**，所对应的应用场景也不同，定时器可看成计数器的一种特例。

由于**精确定时**是定时器/计数器的**最基本功能**，因此，行业内通常将定时器/计数器简称为**定时器**。

STM32定时器的功能

主要功能	高级控制定时器	通用定时器	基本定时器
内部时钟源 (8MHz)	●	●	●
带16位分频的计数单元	●	●	●
更新中断和DMA	●	●	●
计数方向	向上、向下、双向	向上、向下、双向	向上
外部事件计数	●	●	○
其他定时器触发或级联	●	●	○
4个独立输入捕获、输出比较通道	●	●	○
单脉冲输出方式	●	●	○
正交编码器输入	●	●	○
霍尔传感器输入	●	●	○
输出比较信号死区产生	●	○	○
制动信号输入	●	○	○

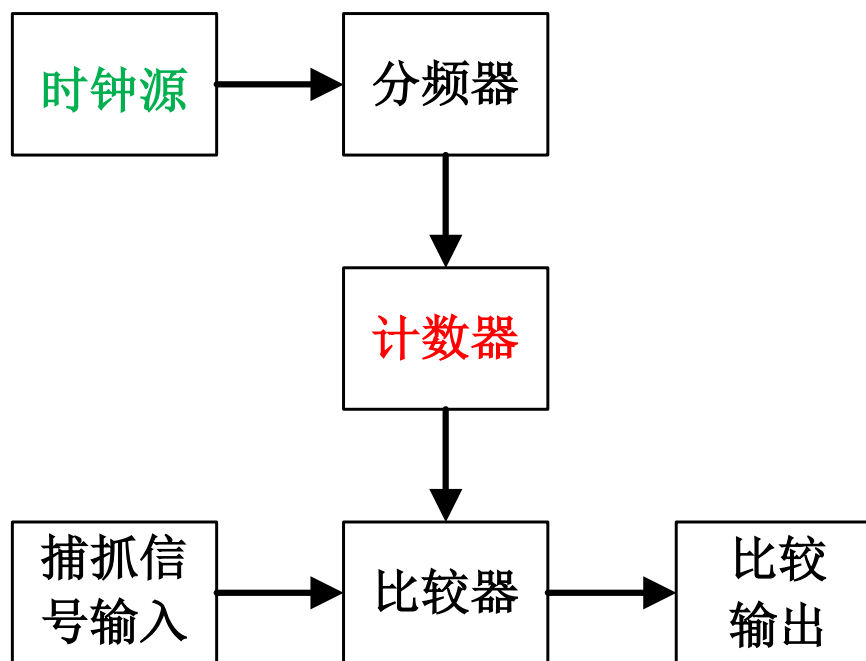


STM32F4系列总共**最多**有14个定时器:

定时器	类 型	计数器长度	预分频系数	计数方向	捕获/比较通道	总线	最大定时器时钟 (MHz)	互补输出	DMA 请求
TIM1 和 TIM8	高级控制	16 位	1~65536 (整 数)	递增、递减、递增/递减	4	APB2	180	有	有
TIM2, TIM5	通用	32 位	1~65536 (整 数)	递增、递减、递增/递减	4	APB1	90/180	无	有
TIM3, TIM4		16 位	1~65536 (整 数)	递增、递减、递增/递减	4			无	有
TIM9		16 位	1~65536 (整 数)	递增	2	APB2	180	无	无
TIM10, TIM11		16 位	1~65536 (整 数)	递增	1			无	无
TIM12		16 位	1~65536 (整 数)	递增	2	APB1	90/180	无	无
TIM13, TIM14		16 位	1~65536 (整 数)	递增	1			无	无
TIM6 和 TIM7		16 位	1~65536 (整 数)	递增	0			无	有

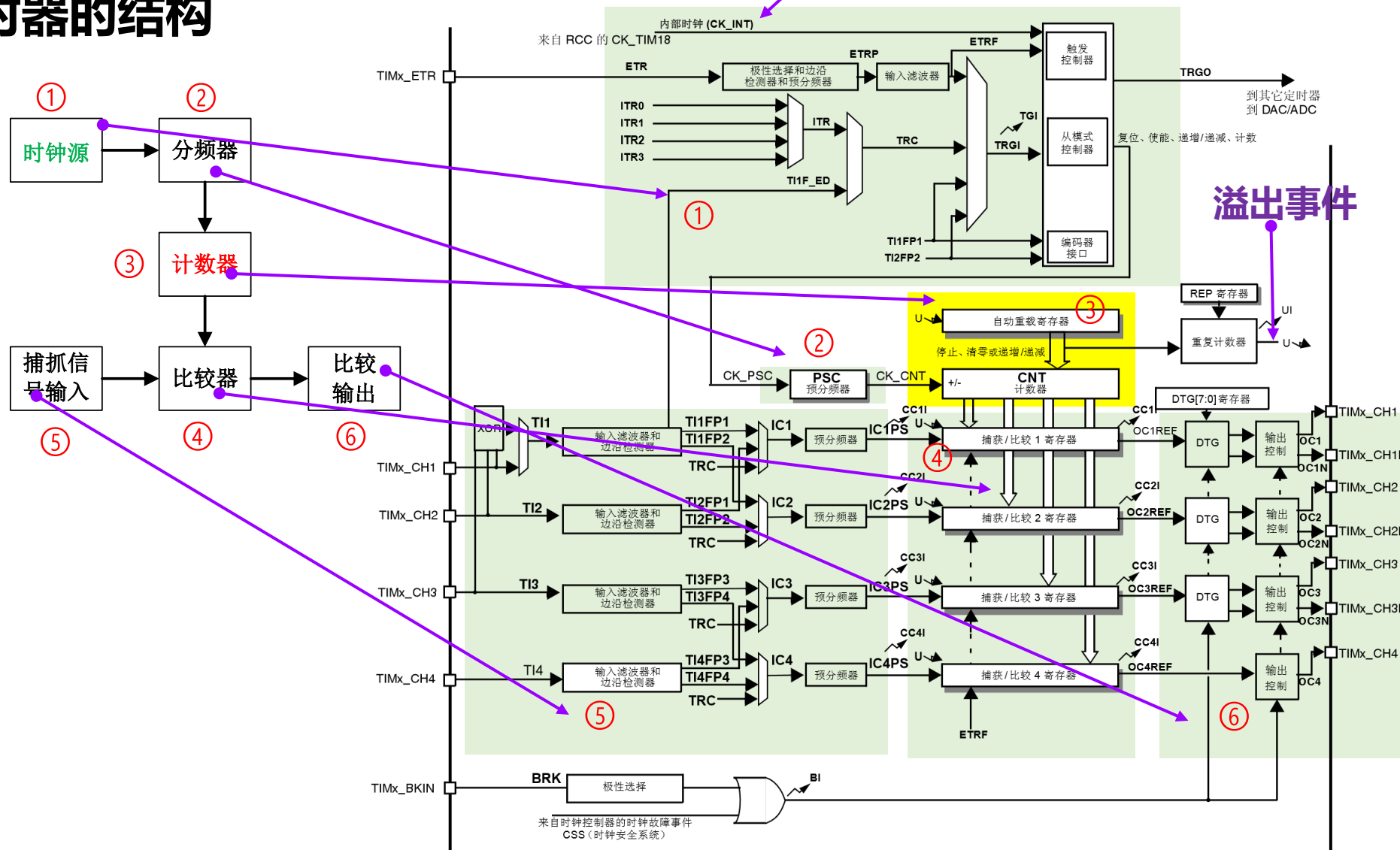
◆ 定时器的结构

定时器可用于各种用途，包括**基本定时功能**、**生成输出波形**（输出比较、PWM 和带死区插入的互补 PWM）和**测量输入信号的脉冲宽度**（输入捕获）等。



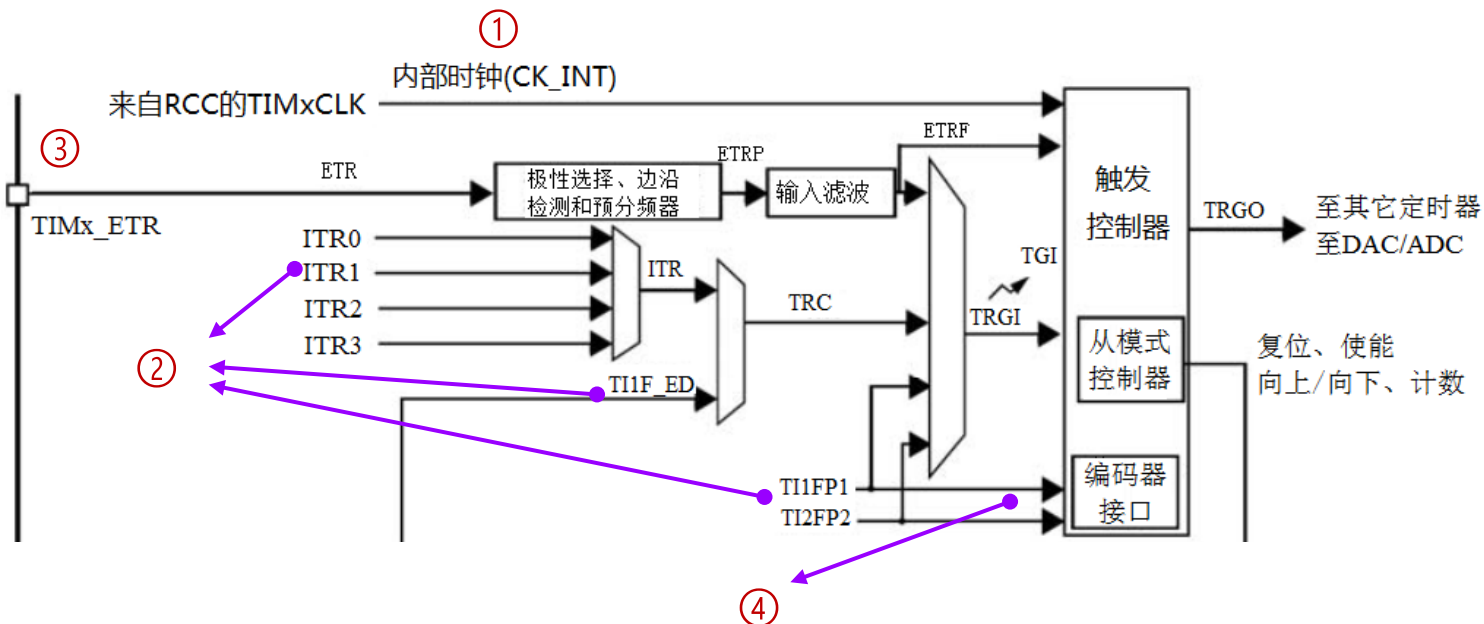
定时器基本结构

内部时钟



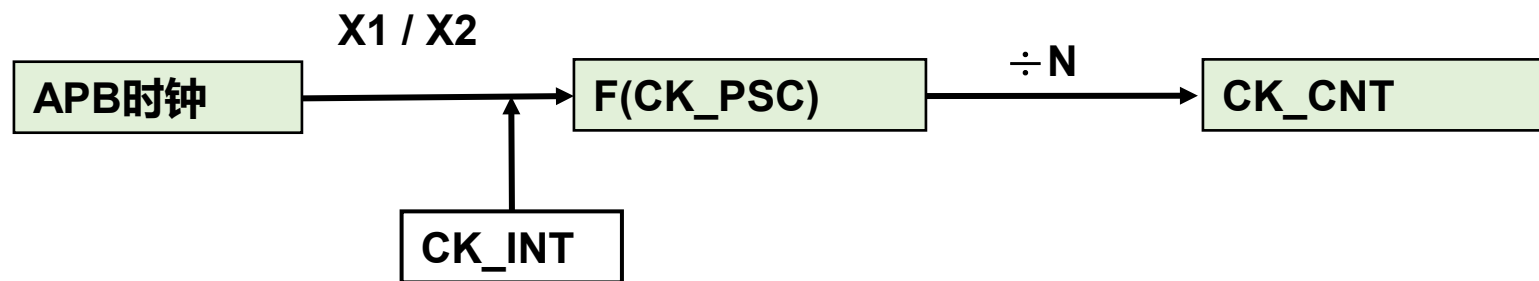
定时器计数需要计数器时钟，可由下列**时钟源**提供：

- ① 内部时钟（CK_INT）源模式。
- ② 外部时钟源模式1：外部输入引脚。
- ③ 外部时钟源模式2：外部触发输入ETR。
- ④ 编码器模式。



◆ 定时器计数时钟源

选择内部时钟



除非APB的分频系数是1，否则**通用定时器的时钟等于APB时钟的2倍**

默认调用SystemInit函数情况下：

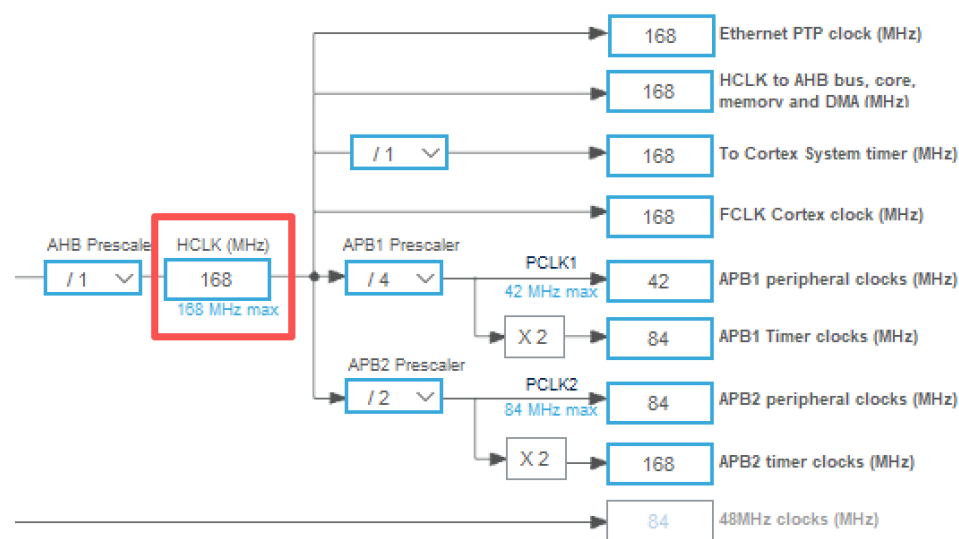
SYSCLK=168M

AHB时钟=168M

APB1时钟=42M， APB2时钟=84M

APB分频系数分别是4和2

定时器时钟是相应挂载总线时钟的2倍

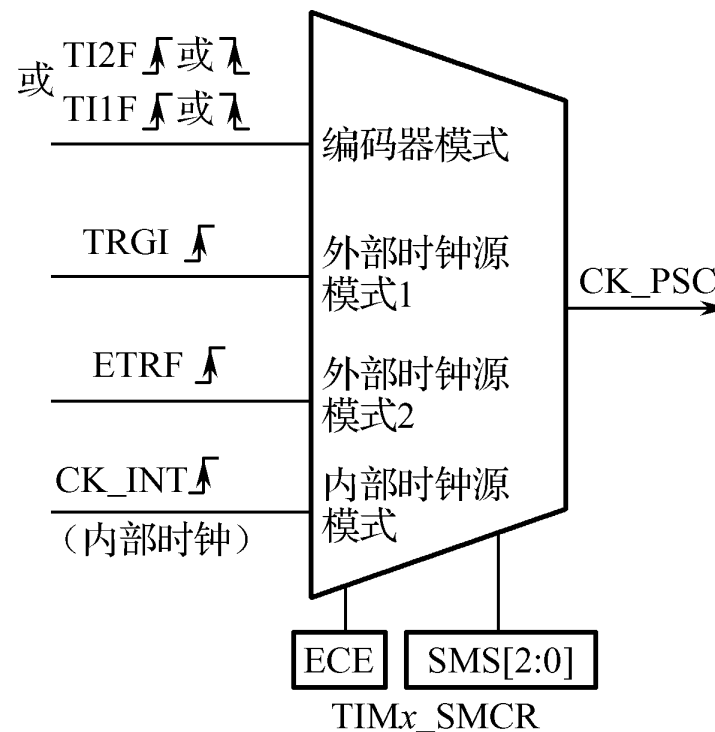


◆ 时钟源

1. 内部时钟源模式

根据RCC_DKCFGR的TIMPRE位的状态，定时器使用的内部时钟源频率不同。

例如，在 $HCLK=168MHz$ ， $PCLK2=84MHz$ （APB2总线时钟预分频系数=2）的情况下，定时器TIM1的内部时钟源 $=2 \times PCLK2=168MHz$ 。

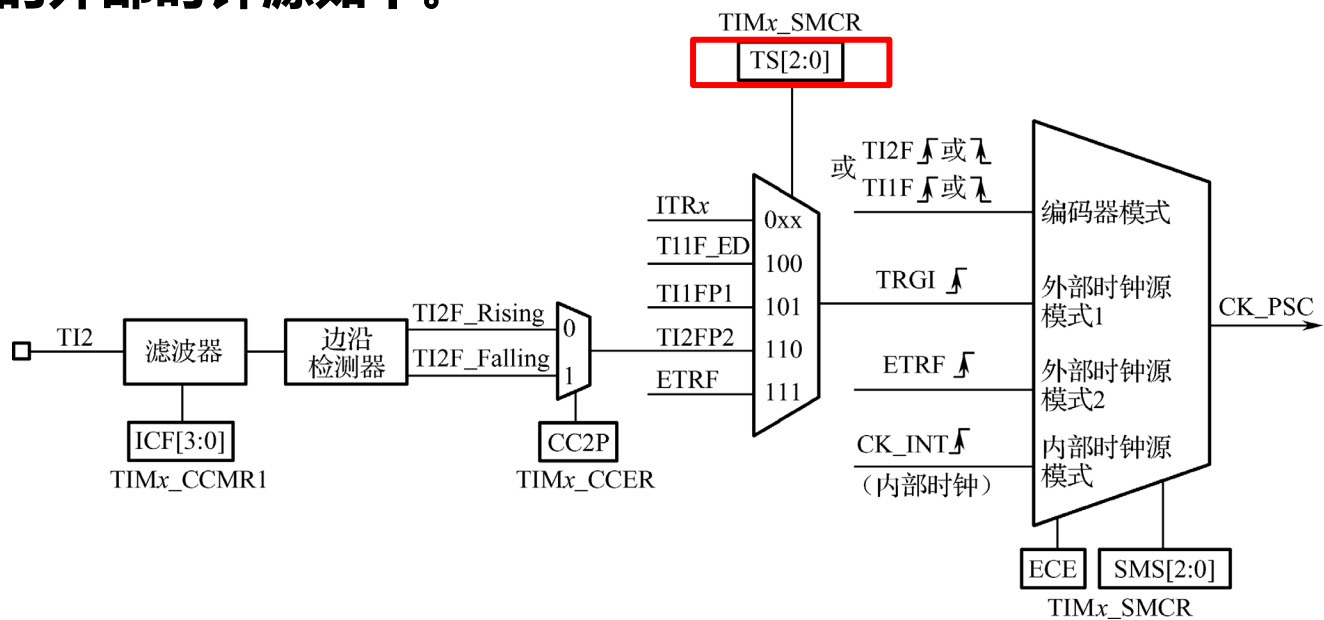


定时器计数时钟源

◆ 时钟源

2. 外部时钟源模式1

计数器可在选定的输入信号上出现上升沿或下降沿时计数。外部时钟源模式1结构如图所示。此时，对于TIMx_SMCR的中位段TS的设置，可选择的外部时钟源如下。

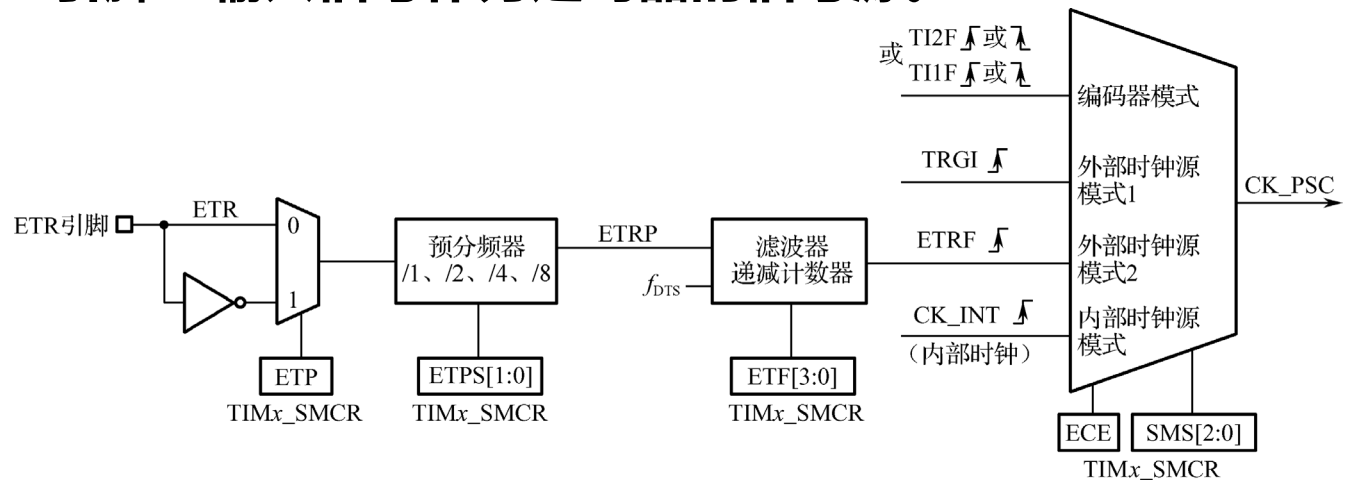


外部时钟源模式1结构图

◆ 时钟源

3. 外部时钟源模式2

以ETR引脚上输入信号作为定时器的信号源。



外部时钟源模式2结构图

4. 编码器模式

将TI1FP1和TI2FP2信号的电平状态的变化作为定时器时钟源。这一模式主要用于测量光电正交编码器输出脉冲数，以及测量电机转速和方向。

03 基本定时功能

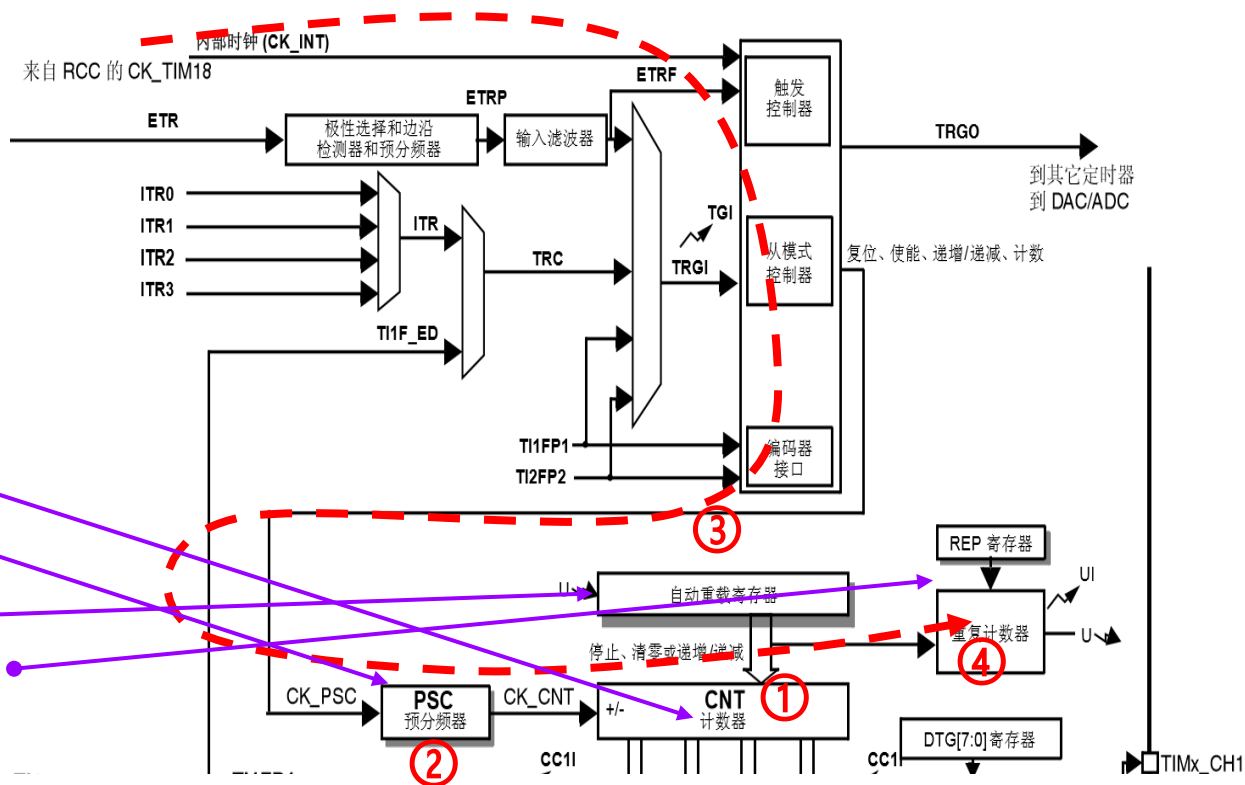
1 时基单元

定时器的主要模块是一个 16 位**计数器**及其相关的**自动重载寄存器**。计数器的时钟可通过**预分频器**进行分频。

时基单元包括：

- ① 计数器寄存器 (TIMx_CNT)
- ② 预分频器寄存器 (TIMx_PSC)
- ③ 自动重载寄存器 (TIMx_ARR)
- ④ 重复计数器寄存器 (TIMx_RCR)

以内部时钟作为计数基准



1 时基单元

计数器、自动重载寄存器和预分频器寄存器可通过软件进行读写。即使在计数器运行时也可执行读写操作。

- ①计数器寄存器 (TIMx_CNT): 计数器, 可递增计数、递减计数或交替进行递增和递减计数;
- ②预分频器寄存器 (TIMx_PSC): 可将计数时间基准在1-65536之间任意分频;
- ③自动重载寄存器 (TIMx_ARR): 存储计数器的溢出值;
- ④重复计数器寄存器 (TIMx_RCR): 高级定时器TIM1、8才有。控制重复计数的次数。

2 计数模式

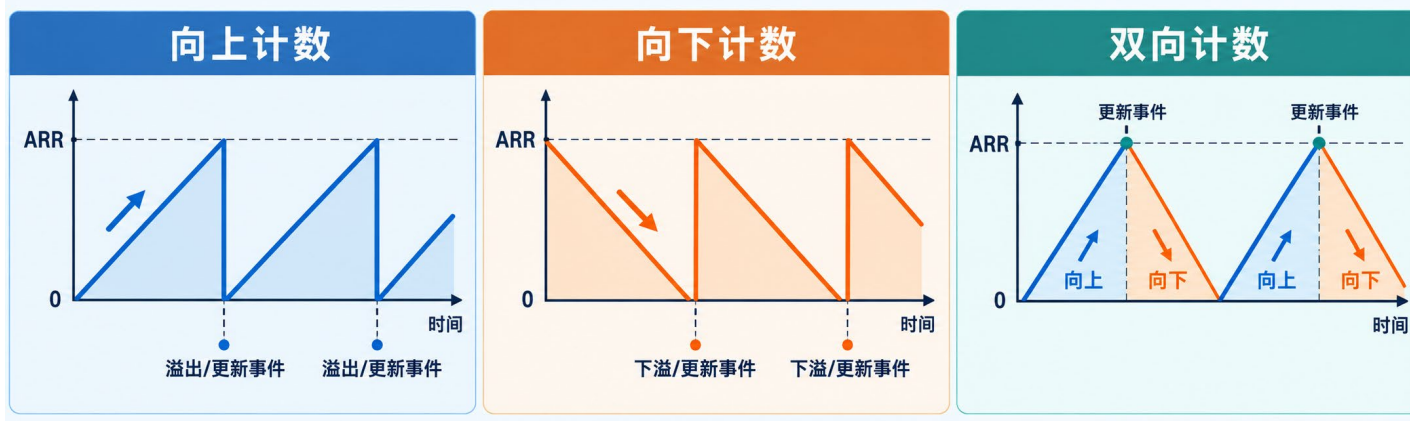
定时器以一定的**时钟基准**进行计数，计数对象是计数寄存器 **TIMx_CNT**。

通用定时器可以向上计数、向下计数、向上向下双向计数模式。

①**递增计数模式**：计数器从0计数到自动加载值(TIMx_ARR)，然后重新从0开始计数并且产生一个计数器**溢出事件**。

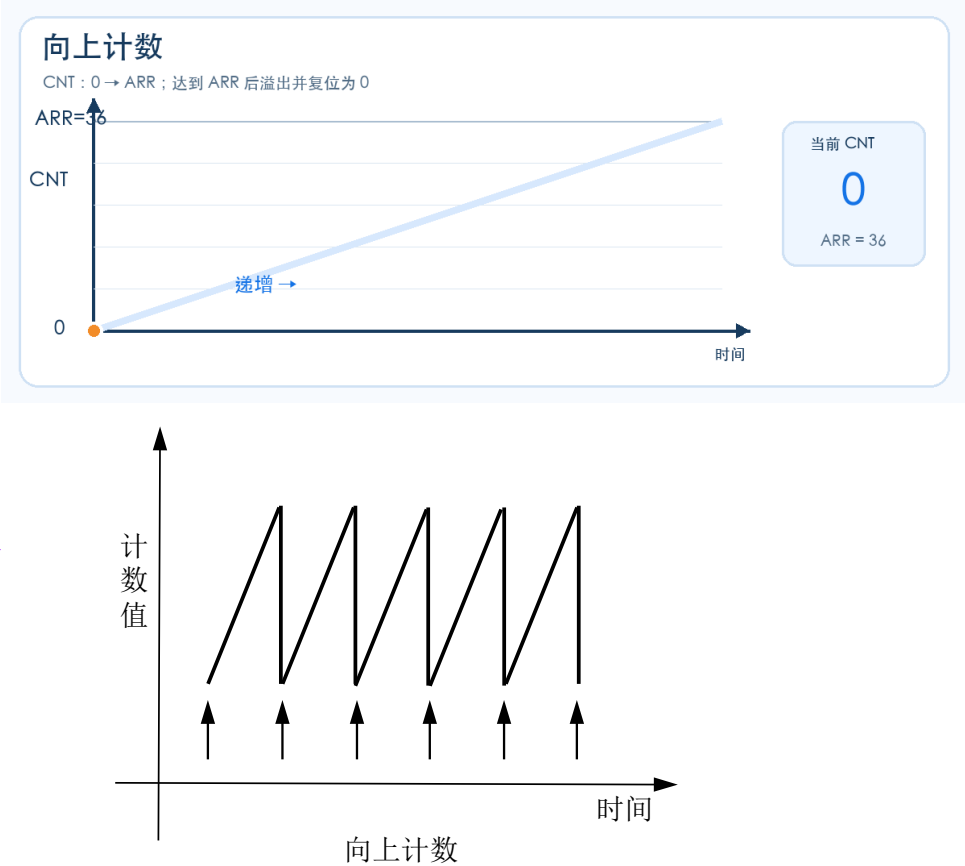
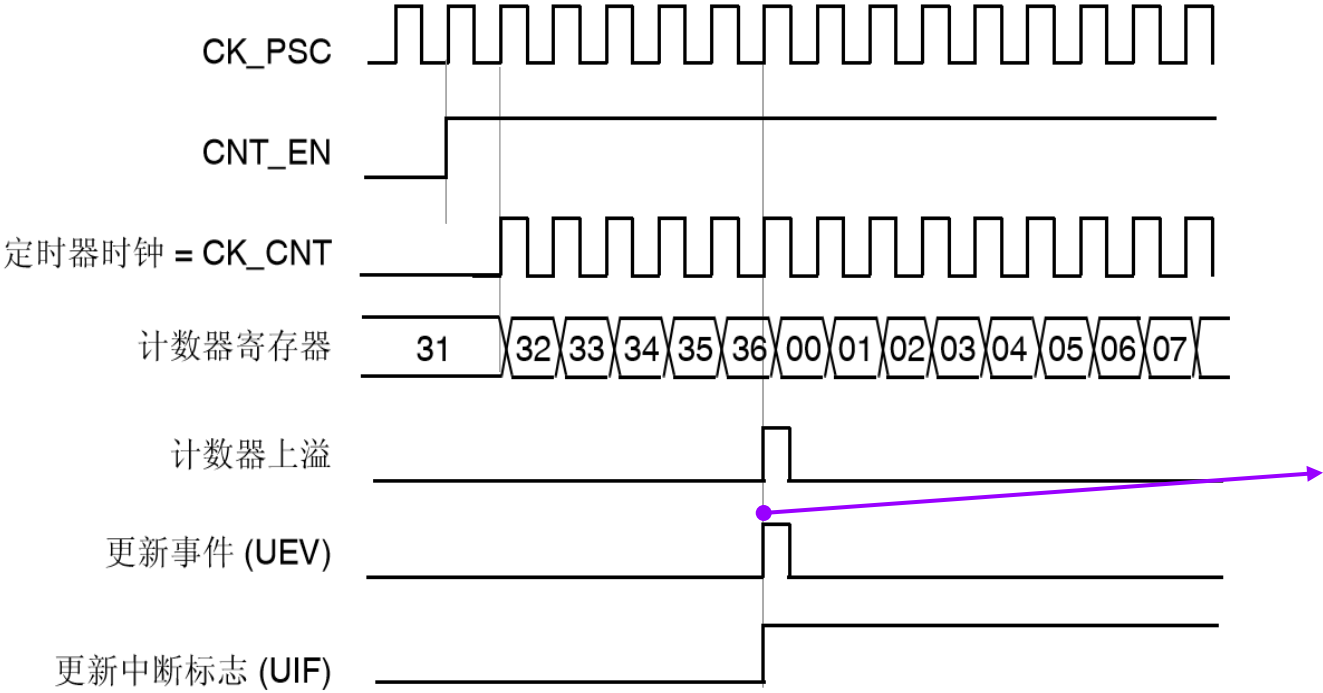
②**递减计数模式**：计数器从自动装入的值(TIMx_ARR)开始向下计数到0，然后从自动装入的值重新开始，并产生一个计数器**向下溢出事件**。

③**中央对齐模式（向上/向下计数）**：计数器从0开始计数到自动装入的值-1，产生一个计数器**溢出事件**，然后向下计数到1并且产生一个计数器**溢出事件**；然后再从0开始重新计数。



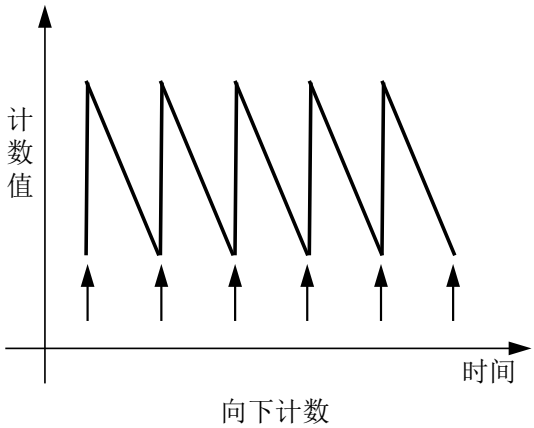
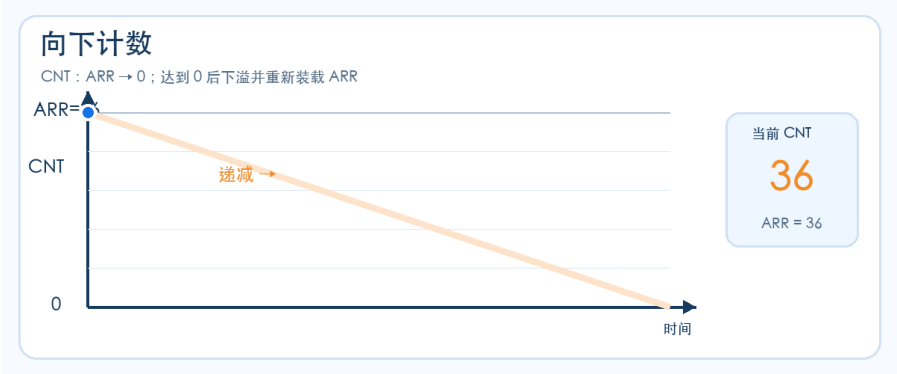
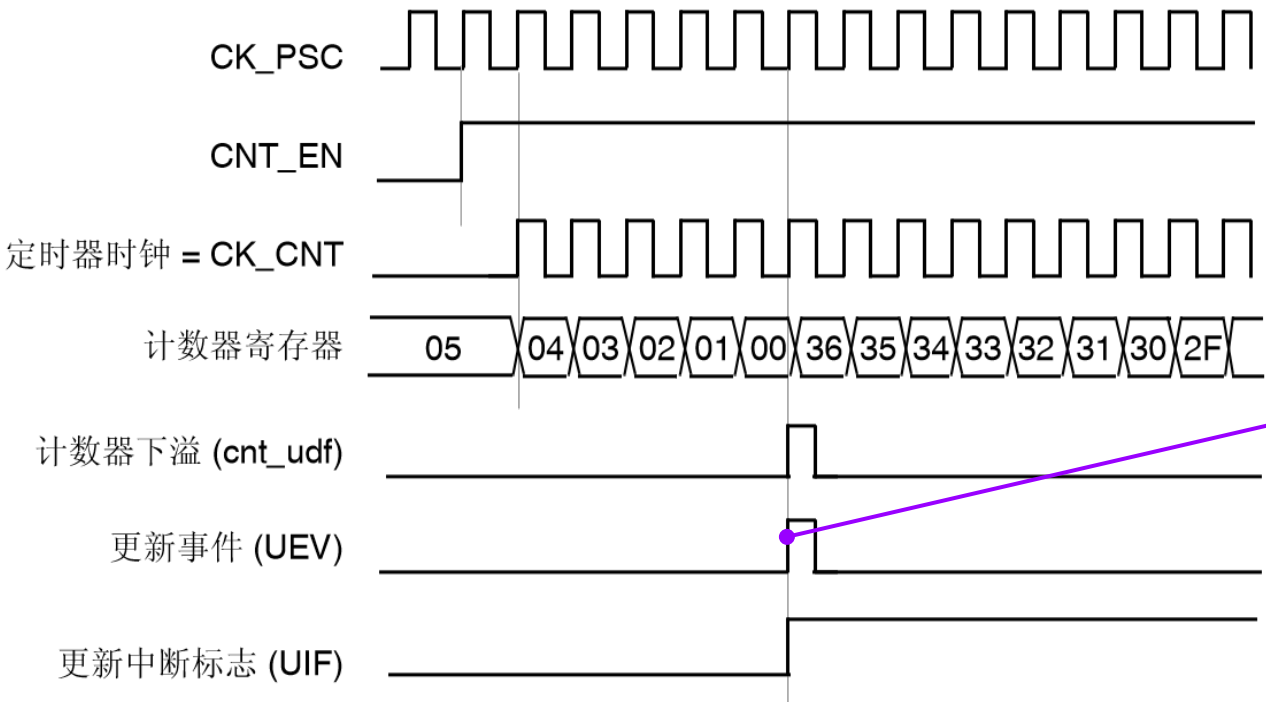
2 计数模式

递增计数模式 (时钟分频因子=1 ARR=36)



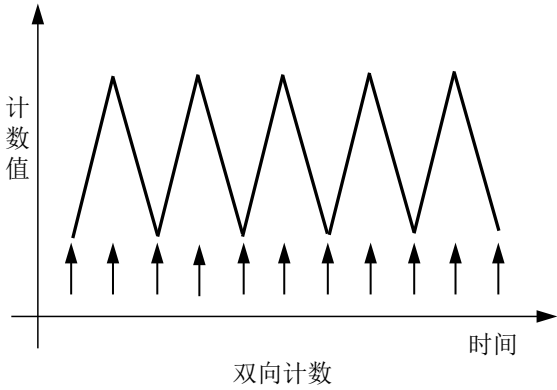
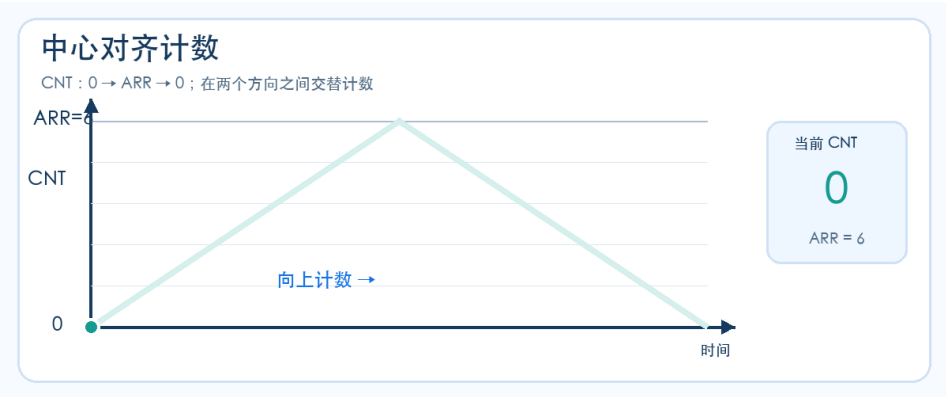
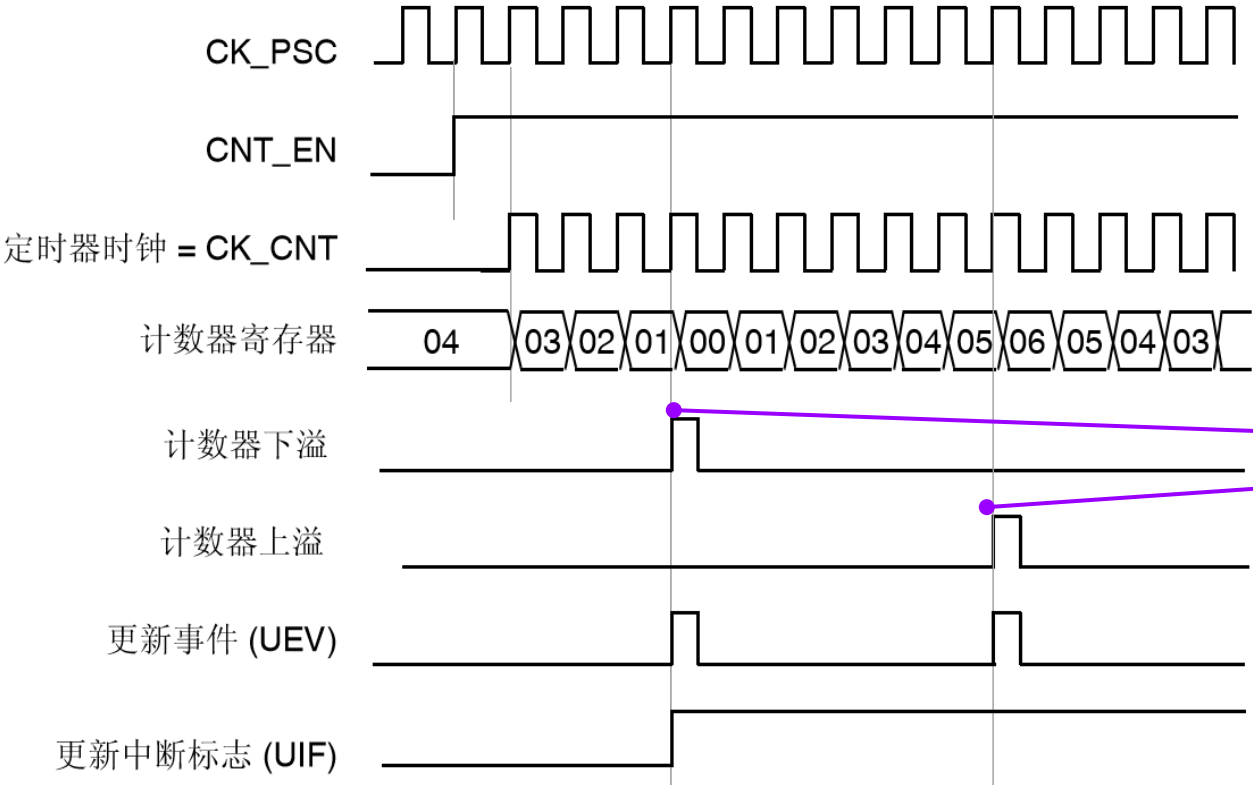
2 计数模式

递减计数模式 (时钟分频因子=1 ARR=36)



2 计数模式

中央对齐 (向上/向下, 双向) 计数模式 (时钟分频因子=1 ARR=6)



04 捕获/比较功能

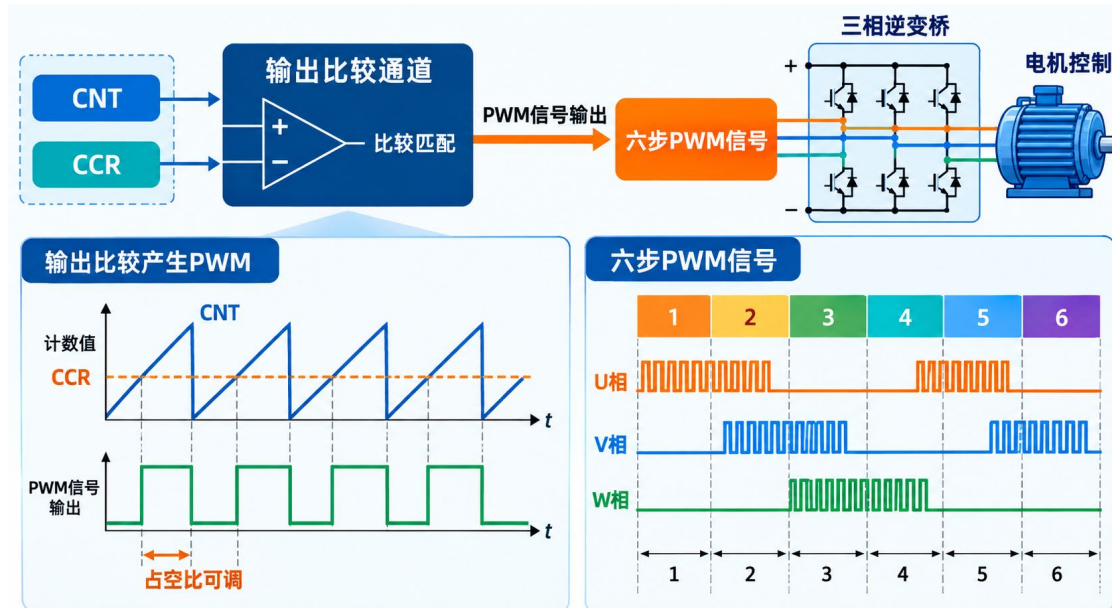
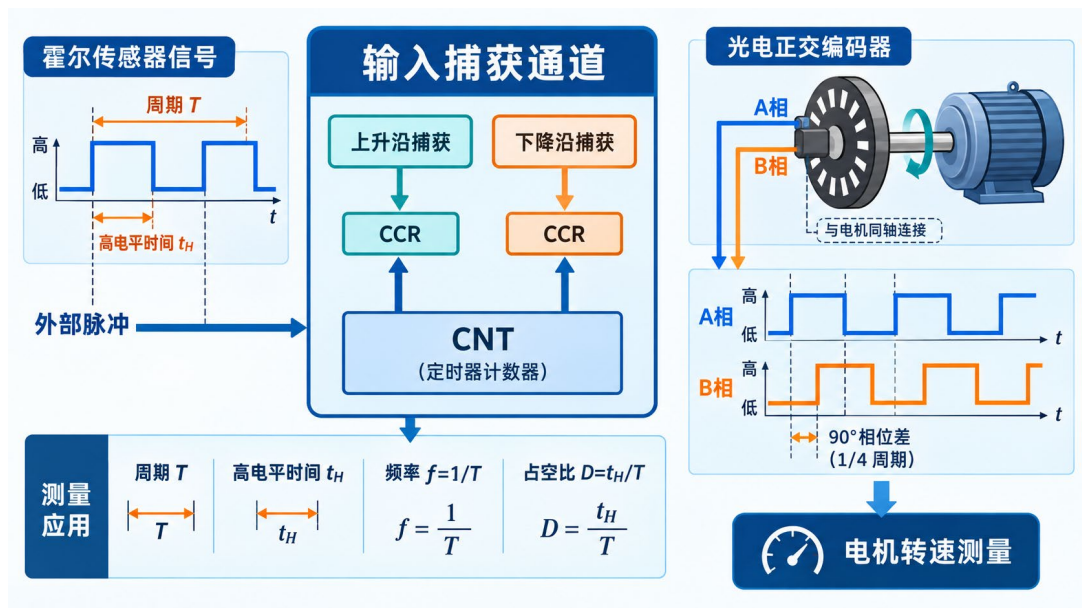
STM32F4微控制器的高级定时器和通用定时器中有**输入捕获通道**和**输出比较通道**。

1、输入捕获通道：

- 1) **频率测量、PWM信号周期、占空比测量，以及霍尔传感器输出信号测量等。**
- 2) **测量光电正交编码器输出信号，实现电机转速的测量。**

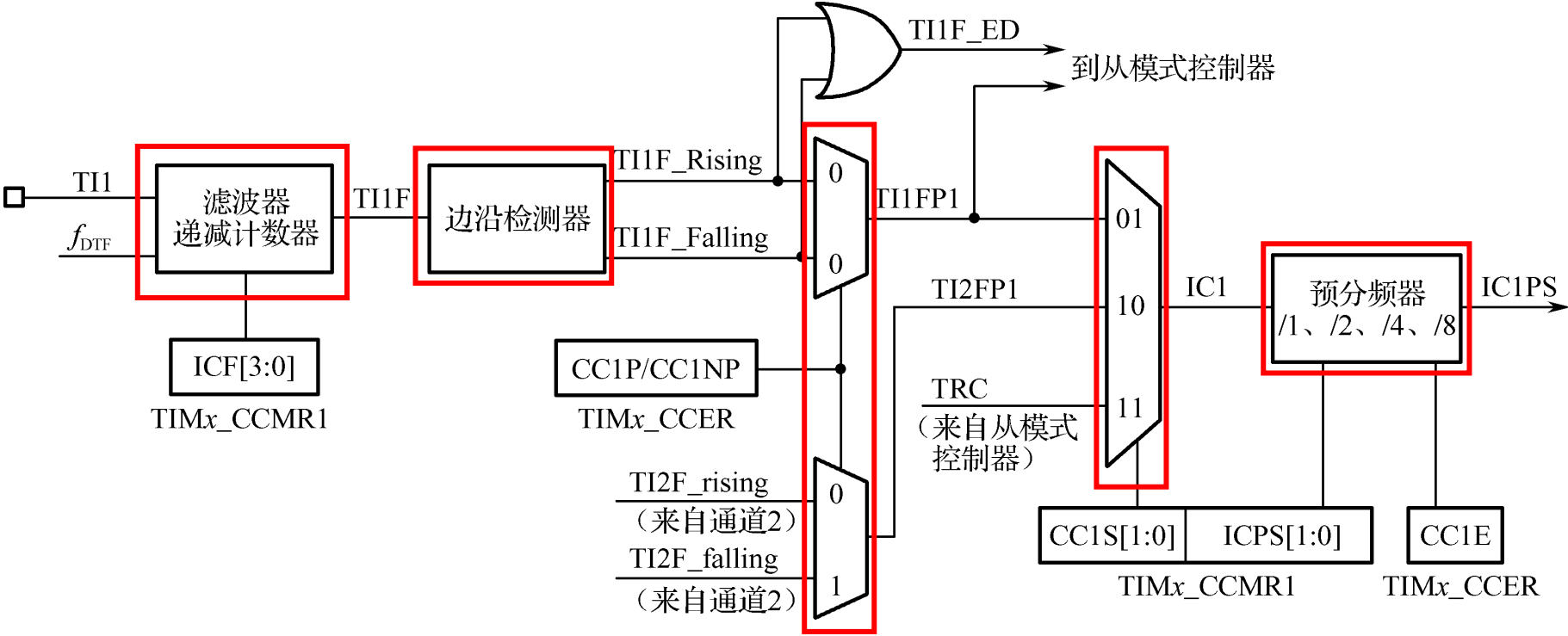
2、输出比较通道：

实现**PWM信号**输出、6步PWM信号生成，用于电机控制。



1 输入捕获/比较输出通道

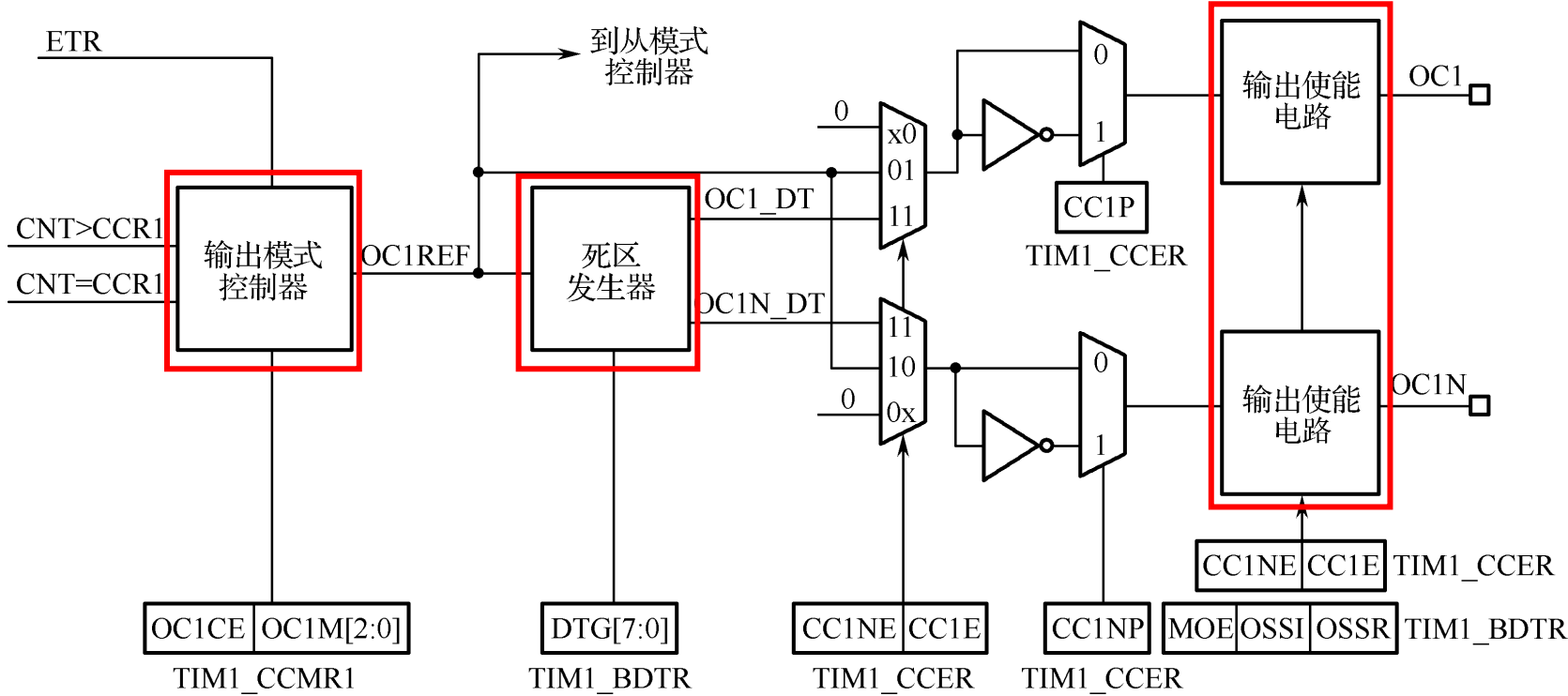
输入捕获通道由**输入滤波器**（防止去除输入信号电平切换产生的抖动，防止误判）、**边沿检测器**、**边沿检测方式选择开关**（2选1）、**输入捕抓信号选择多路开关**（3选1）及**预分频器**组成。



输入捕获通道结构示意图

1 输入捕获/输出比较通道

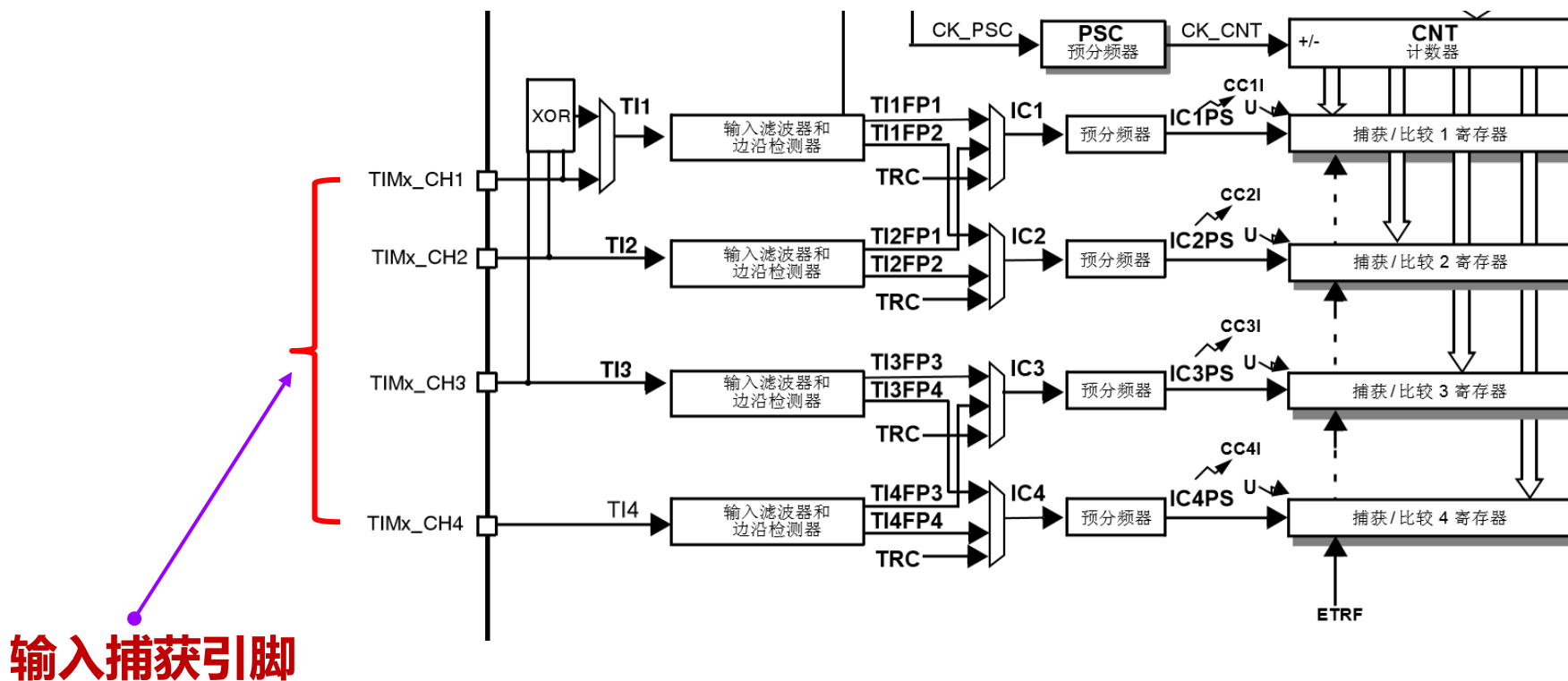
输出比较通道由**输出模式控制器**、**死区发生器**（防止电动机驱动上下桥臂控制的开关状态切换时出现同时导通）和**输出使能控制电路**组成。



输出比较通道结构示意图

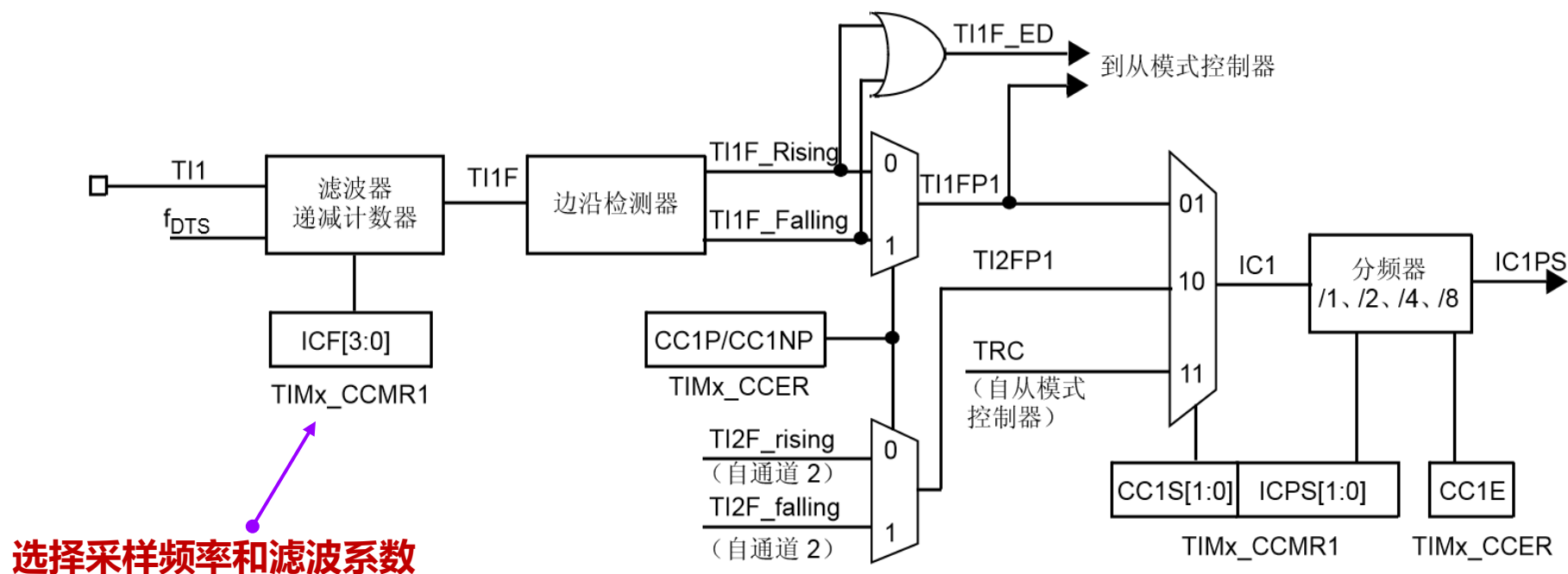
2 输入捕获模式

在输入捕获模式下，当检测到捕获输入通道信号的跳变后，**将使用捕获/比较寄存器 (TIMx_CCRx) 来锁存计数器的值**。发生捕获事件时，会将相应的 CCxIF 标志 (TIMx_SR 寄存器) 置 1，并可发送中断或 DMA 请求（如果已使能）。



2 输入捕获模式

在输入捕获模式下，当检测到捕获输入通道信号的跳变后，**将使用捕获/比较寄存器 (TIMx_CCRx) 来锁存计数器的值**。发生捕获事件时，会将相应的 CCxIF 标志 (TIMx_SR 寄存器) 置 1，并可发送中断或 DMA 请求（如果已使能）。



2 输入捕获模式

在使用输入捕获通道1，当检测到TI1引脚上输入的信号出现上升沿时，将计数器的值捕获到TIMx_CCR1中。具体操作步骤如下：

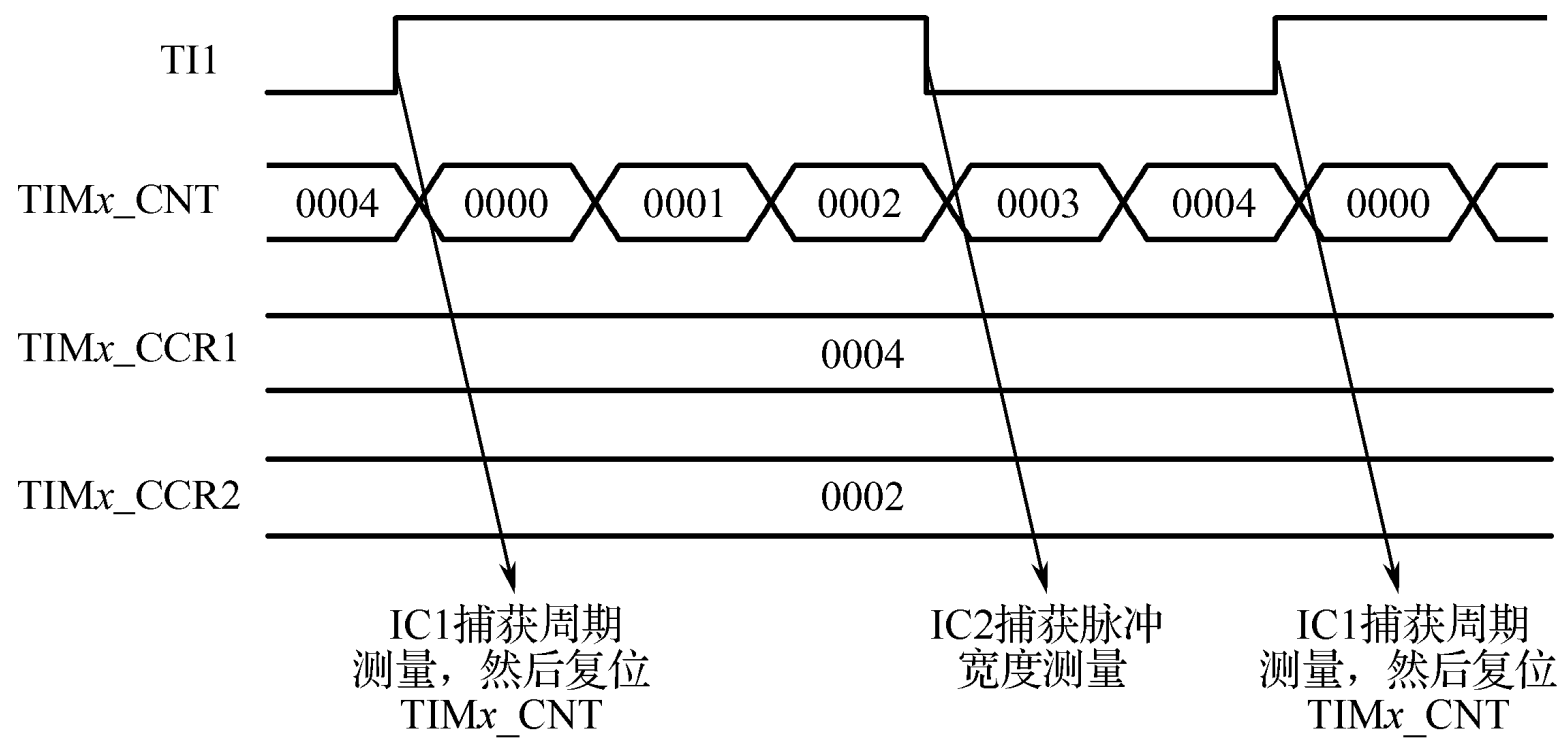
1. 选择输入捕获模式，IC1映射到TI1上
2. 设定输入信号边沿检测的滤波功能（防抖动）
3. 选择边沿触发方式
4. 对输入预分频器进行编程
5. 使能输入捕获功能
6. 设置捕获中断和DMA请求

当**连续两次**捕获同一输出信号的连续两个边沿跳变，两次得到的计数寄存器值分别为**C1和C2**（假设C1和C2计数期间没有溢出事件），那么这一**输入信号的周期**：

$$\left((C2 - C1) / CK_CNT \right) / \text{输入捕获通道预分频系数}$$

2 输入捕获模式

方波信号的周期测量:



方波信号的周期测量示意图

3 PWM输出模式

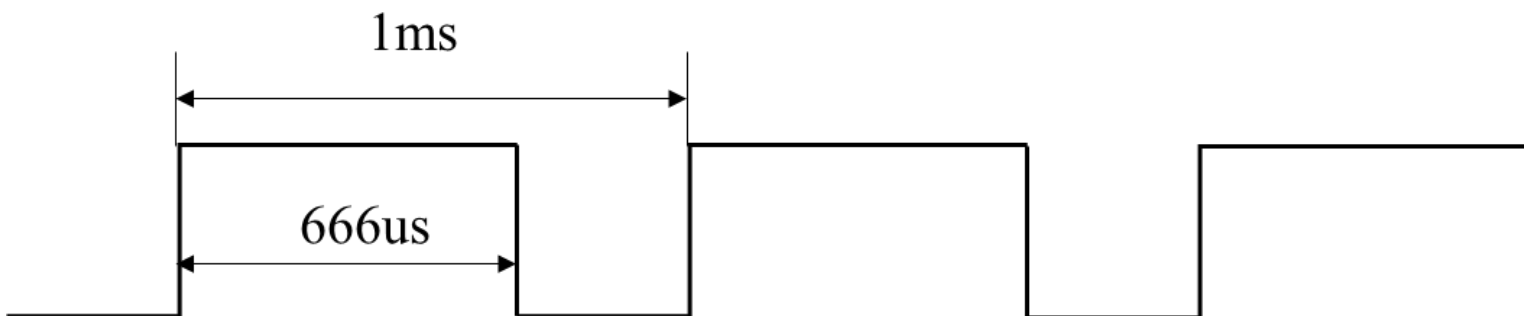
脉冲宽度调制 (Pulse-Width Modulation, PWM) 简称脉宽调制，是一种对模拟信号电平进行数字编码的方法，可以利用MCU的数字输出，通过对一系列脉冲宽度进行调制，实现对模拟电路的控制，广泛应用于测量、通信、工控等领域。

1. PWM参数

PWM信号具有三个基本参数，即**周期**、**频率**和**占空比**。

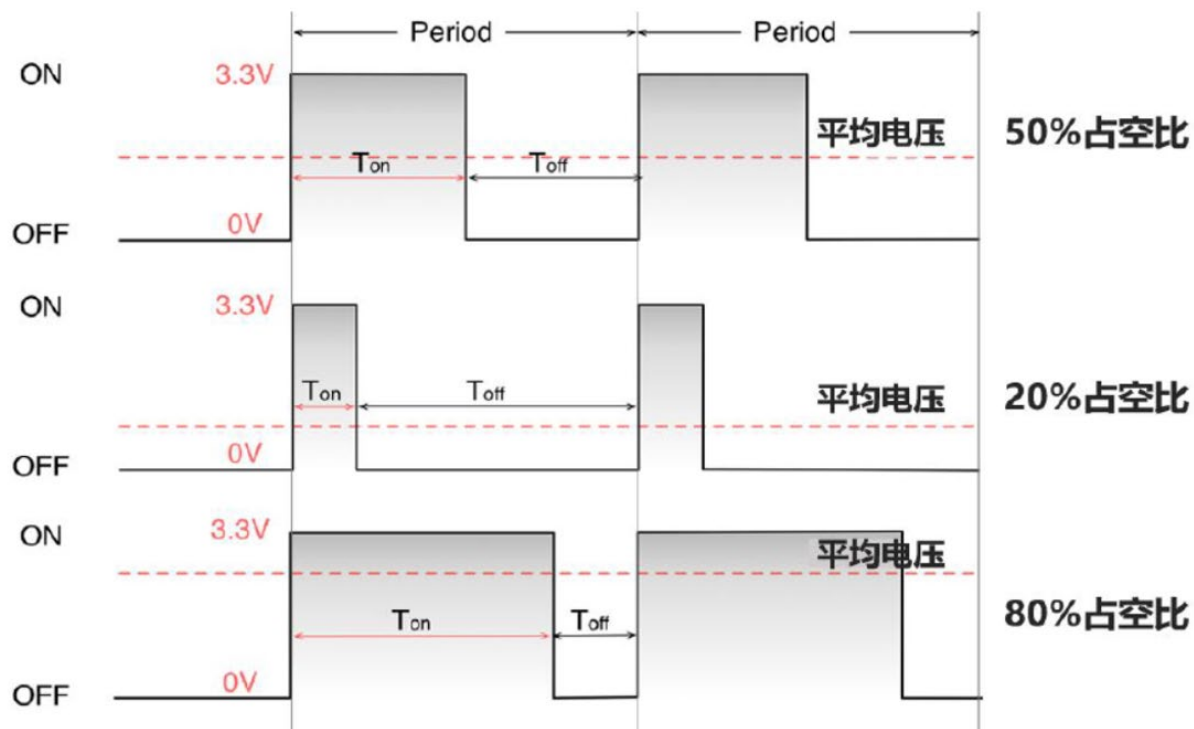
周期是一个完整的PWM波形所持续的时间；频率是周期的倒数，即1s内完整PWM波形的个数，单位为Hz；

占空比是高电平持续时间与周期之比，用百分比表示。



3 PWM输出模式

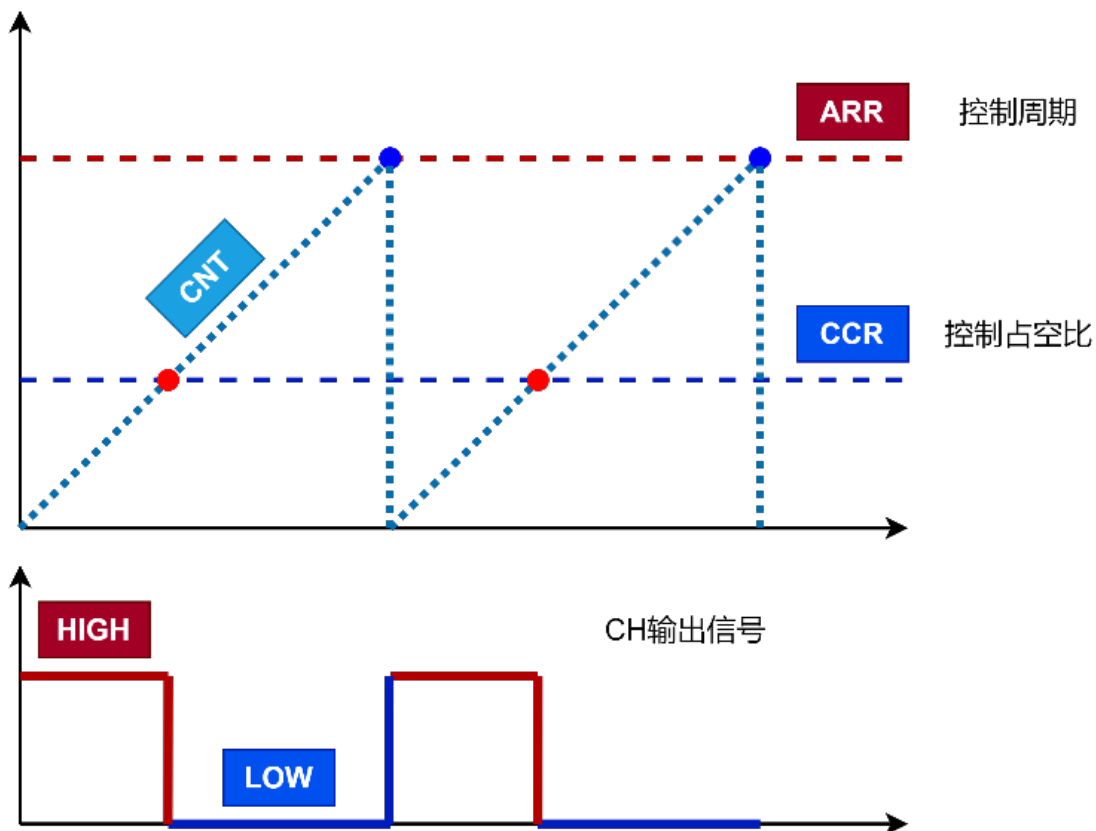
PWM对模拟信号编码的本质是将脉冲信号加到模拟负载，高电平时提供直流输出，低电平时断开直流输出。理论上，通过对高电平和低电平的时间控制，可以输出任意不大于高电平电压的模拟电压，**输出电压为高电平电压与占空比的乘积。**



3 PWM输出模式

STM32输出PWM原理

STM32通用定时器和高级定时器具有PWM输出功能，通过设置**预分频器（PSC）**、**自动重载寄存器（ARR）**和**捕获比较寄存器（CCR）**值可获得任意周期（频率）和占空比的PWM输出。



PSC和ARR用于控制PWM的周期，CCR用于控制PWM的占空比。

当周期确定后，每个周期CNT值加1，并与CCR值比较，当CNT值小于CCR值时，CH通道输出高电平，当CNT值大于CCR值时，CH通道输出低电平，当CNT值等于ARR值时，CNT值自动置零，重新开始计数并与CCR值进行比较，进而实现连续的PWM输出。

3 PWM输出模式

CH输出信号与计数方向、PWM模式和输出极性的关系

CH通道输出信号与**计数方向**、**PWM模式**和**输出极性**均有关系。

输出极性包括高和低两种极性，决定了CH输出的有效电平，如极性为高则有效电平为高电平，反之有效电平为低电平。

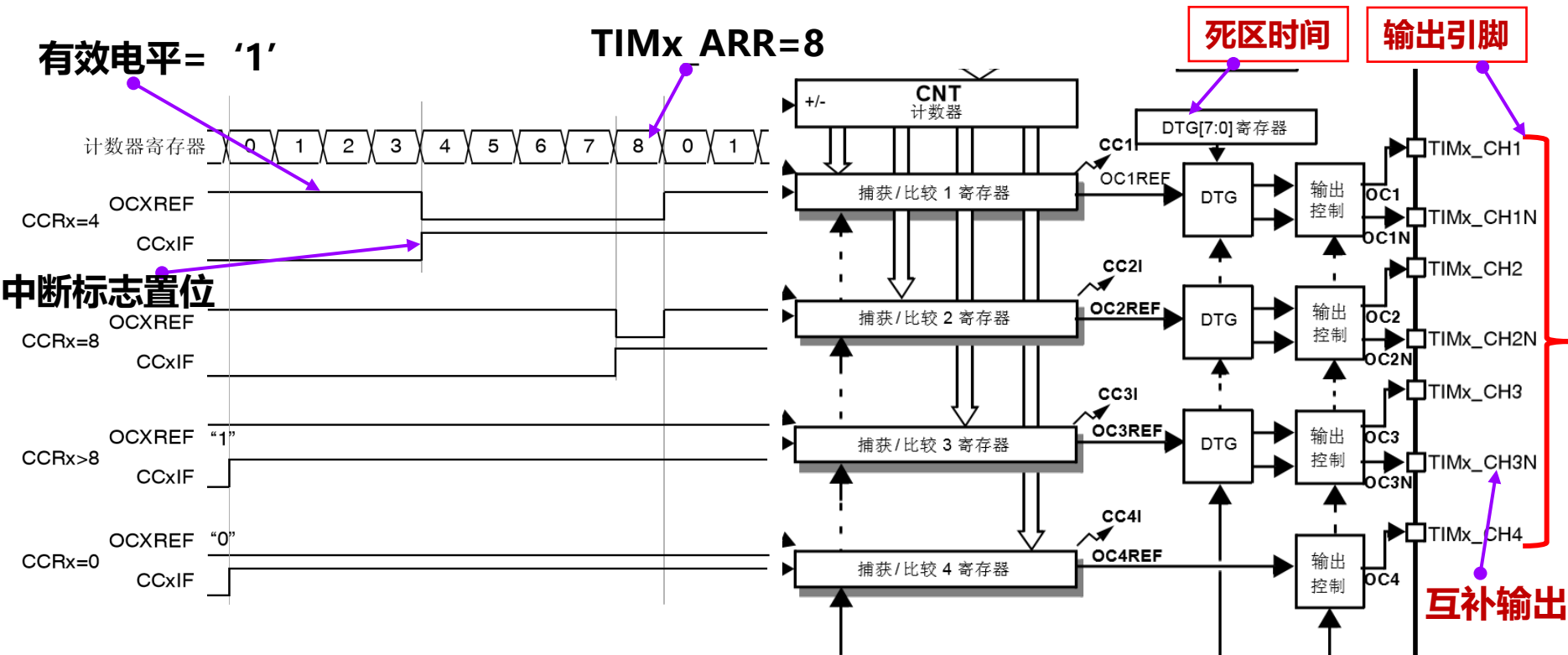
当计数方向为向上或向下计数时，PWM有两种模式，即PWM模式一和PWM模式二。

计数方向	CNT与CCR	PWM模式	
		PWM模式一	PWM模式二
向上计数	CNT < CCR	有效电平	无效电平
	CNT > CCR	无效电平	有效电平
向下计数	CNT < CCR	无效电平	有效电平
	CNT > CCR	有效电平	无效电平

3 PWM输出模式

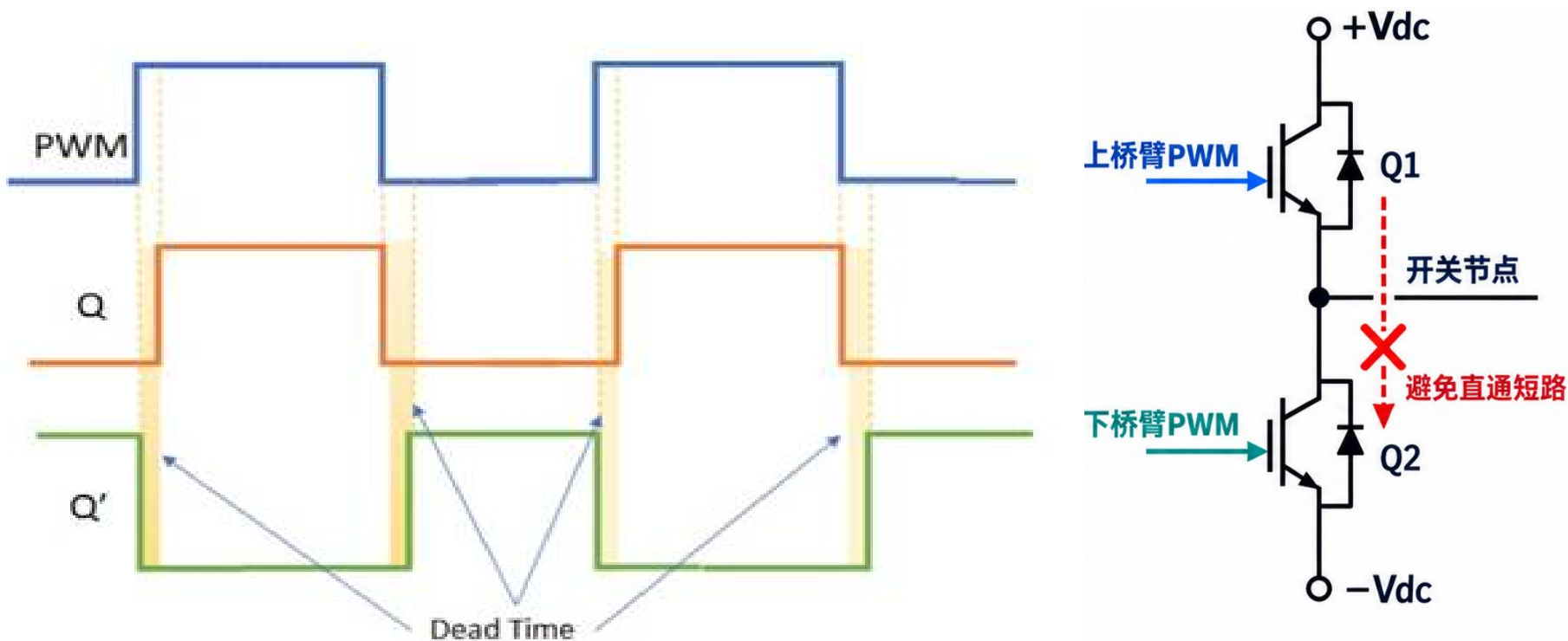
设定当使能比较输出功能 (TIMx_CCMR2x) 时, 可以通过相关比较输出引脚输出PWM波。

自动重载寄存器 (TIMx_ARR) 设定PWM波的**周期**, 捕获/比较寄存器 (TIMx_CCRx) 设定**占空比**。计数器寄存器 (TIMx_CNT)计数值 < TIMx_CCRx的值时, 输出**有效电平**。



3 PWM输出模式

什么是死区 (Dead Zone) ???



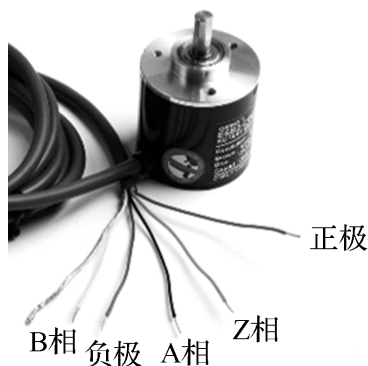
PWM死区：互补PWM切换时，人为插入的短暂关断间隔。

4 编码器接口模式

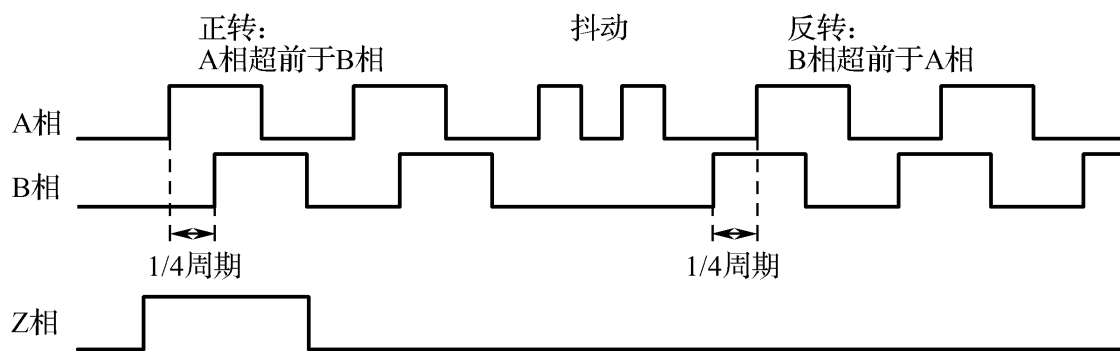
编码器接口主要用于连接正交编码器及**测量电机的转速和转向**。

光电编码器内部的LED发射的光，通过光栅到达光敏管，引起A相和B相电平变化。**如果正转**，则A相输出超前B相90°；**如果反转**，则A相滞后B相90°。每转一周，Z相经过LED一次，输出一个脉冲，可作为编码器的机械零位。

STM32F4微控制器的高级定时器**TIM1和TIM8**，通用定时器**TIM2 ~ TIM5**集成了编码器接口功能。



(a) 光电编码器外观



(b) 正交光电编码器输出波形

常用光电编码器

4 编码器接口模式

在编码器模式下，计数器会根据**增量编码器的速度和方向自动进行修改**，因此计数器内容始终表示编码器的位置。计数方向对应于定时器所连传感器的轴旋转方向。

不同编码器模式下的计数方式

有效边沿	相反信号的电平（TI1FP1对应TI2， TI2FP2对应TI1）	TI1FP1信号		TI2FP2信号	
		上升	下降	上升	下降
仅在TI1处计数	高	递减	递增	不计数	不计数
	低	递增	递减	不计数	不计数
仅在TI2处计数	高	不计数	不计数	递增	递减
	低	不计数	不计数	递减	递增
在TI1和TI2处均计数	高	递减	递增	递增	递减
	低	递增	递减	递减	递增

4 编码器接口模式

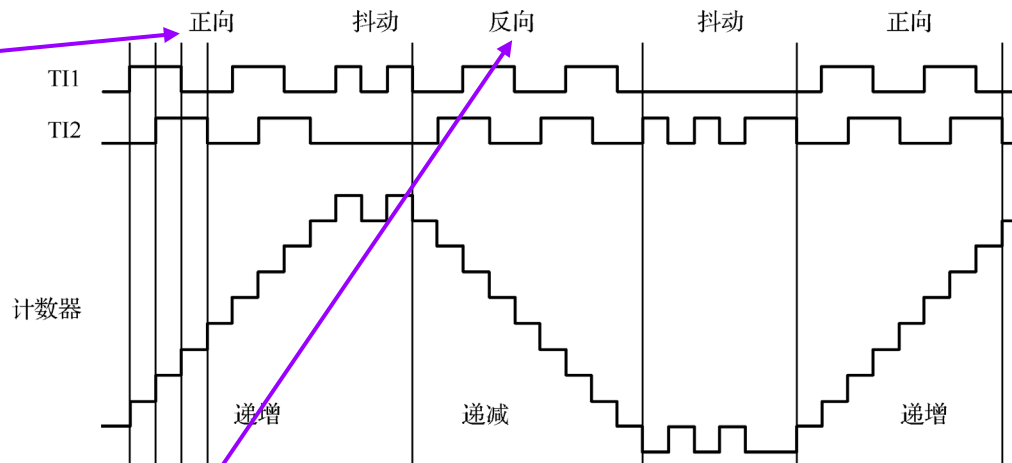
(1) 正转时，计数器递增计数。

TI1高电平时，TI2上升沿处，计数器递增计数。

TI1低电平时，TI1下降沿处，计数器递增计数。

TI2高电平时，TI2下降沿处，计数器递增计数。

TI2低电平时，TI1上升沿处，计数器递增计数。



编码器接口模式，在TI1和TI2处均计数示意图

(2) 反转时，计数器递减计数。

TI1高电平时，TI2下降沿处，计数器递减计数。

TI1低电平时，TI1上升沿处，计数器递减计数。

TI2高电平时，TI2上升沿处，计数器递减计数。

TI2低电平时，TI1下降沿处，计数器递减计数。

头文件: `stm32f4xx_tim.h`

库函数: `stm32f4xx_tim.c`

1、定时器定时周期初始化函数:

```
void TIM_TimeBaseInit(TIM_TypeDef* TIMx,  
    TIM_TimeBaseInitTypeDef* TIM_TimeBaseInitStruct);
```

```
#define TIM2      ((TIM_TypeDef *) TIM2_BASE)  
#define TIM3      ((TIM_TypeDef *) TIM3_BASE)  
#define TIM4      ((TIM_TypeDef *) TIM4_BASE)
```

```
typedef struct  
{  
    uint16_t TIM_Prescaler;    //预分频系数  
    uint16_t TIM_CounterMode; //计数模式  
    uint16_t TIM_Period;      //计数长度  
    uint16_t TIM_ClockDivision; //与死区长度及捕抓采样频率频率相关  
    uint8_t TIM_RepetitionCounter; //重复计数次数  
} TIM_TimeBaseInitTypeDef;
```

```
TIM_TimeBaseStructure.TIM_Period = 4999;  
TIM_TimeBaseStructure.TIM_Prescaler = 7199;  
TIM_TimeBaseStructure.TIM_ClockDivision = TIM_CKD_DIV1;  
TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;  
TIM_TimeBaseInit(TIM3, &TIM_TimeBaseStructure);
```

2、定时器使能函数:

```
void TIM_Cmd(TIM_TypeDef* TIMx, FunctionalState NewState)
```

使能或禁止定时器计数。

3、定时器中断使能函数:

```
void TIM_ITConfig(TIM_TypeDef* TIMx, uint16_t TIM_IT,  
                  FunctionalState NewState);
```

使能或禁止定时器中断。

4、状态标志位获取和清除

//获取需要的定时器状态标志位

```
FlagStatus TIM_GetFlagStatus(TIM_TypeDef* TIMx, uint16_t TIM_FLAG);
```

```
Void TIM_ClearFlag(TIM_TypeDef* TIMx, uint16_t TIM_FLAG);
```

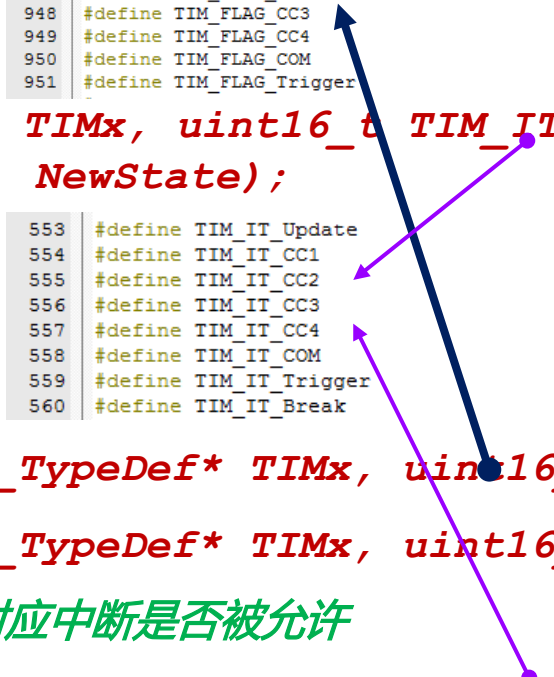
//获取需要的定时器状态标志位, 并且判断对应中断是否被允许

```
ITStatus TIM_GetITStatus(TIM_TypeDef* TIMx, uint16_t TIM_IT);
```

```
void TIM_ClearITPendingBit(TIM_TypeDef* TIMx, uint16_t TIM_IT);
```

```
945 #define TIM_FLAG_Update      ((uint16_t)0x0001)  
946 #define TIM_FLAG_CC1         ((uint16_t)0x0002)  
947 #define TIM_FLAG_CC2         ((uint16_t)0x0004)  
948 #define TIM_FLAG_CC3         ((uint16_t)0x0008)  
949 #define TIM_FLAG_CC4         ((uint16_t)0x0010)  
950 #define TIM_FLAG_COM         ((uint16_t)0x0020)  
951 #define TIM_FLAG_Trigger      ((uint16_t)0x0040)
```

```
553 #define TIM_IT_Update         ((uint16_t)0x0001)  
554 #define TIM_IT_CC1           ((uint16_t)0x0002)  
555 #define TIM_IT_CC2           ((uint16_t)0x0004)  
556 #define TIM_IT_CC3           ((uint16_t)0x0008)  
557 #define TIM_IT_CC4           ((uint16_t)0x0010)  
558 #define TIM_IT_COM           ((uint16_t)0x0020)  
559 #define TIM_IT_Trigger        ((uint16_t)0x0040)  
560 #define TIM_IT_Break         ((uint16_t)0x0080)
```



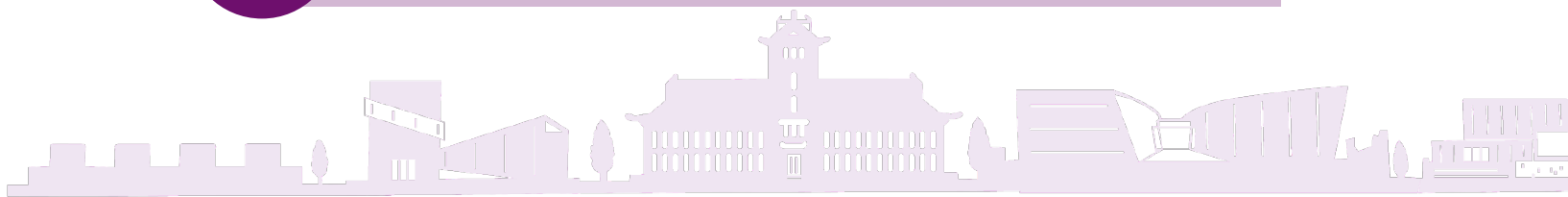
本章小结

01 STM32的时钟系统

02 定时器系统简介

03 基本定时功能

04 捕获/比较功能



谢谢！